CEFET/RJ

TÉCNICAS DE APRENDIZADO DE MÁQUINA

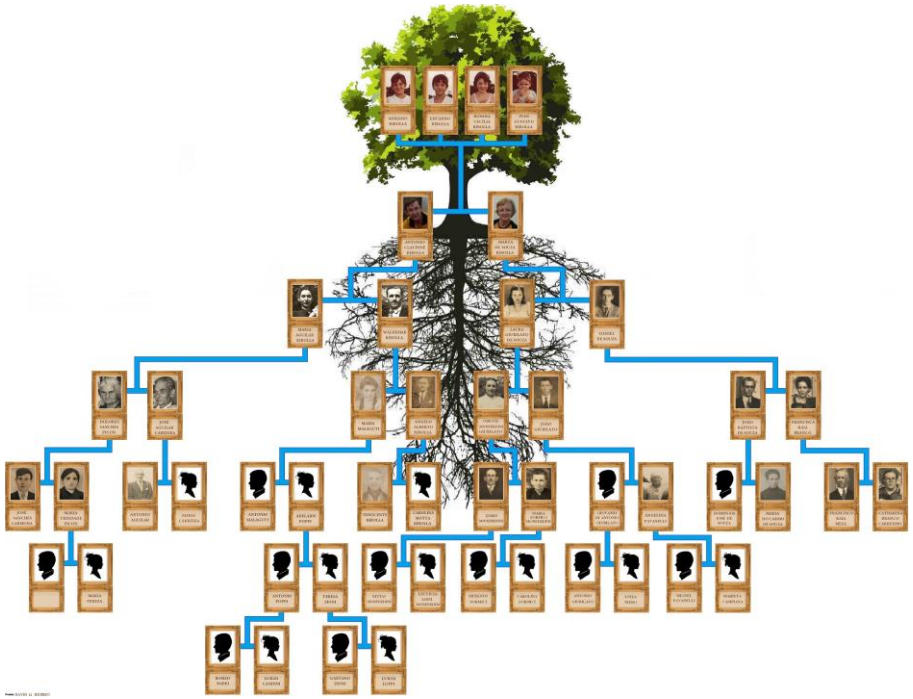
Bacharelado em Sistemas da Informação

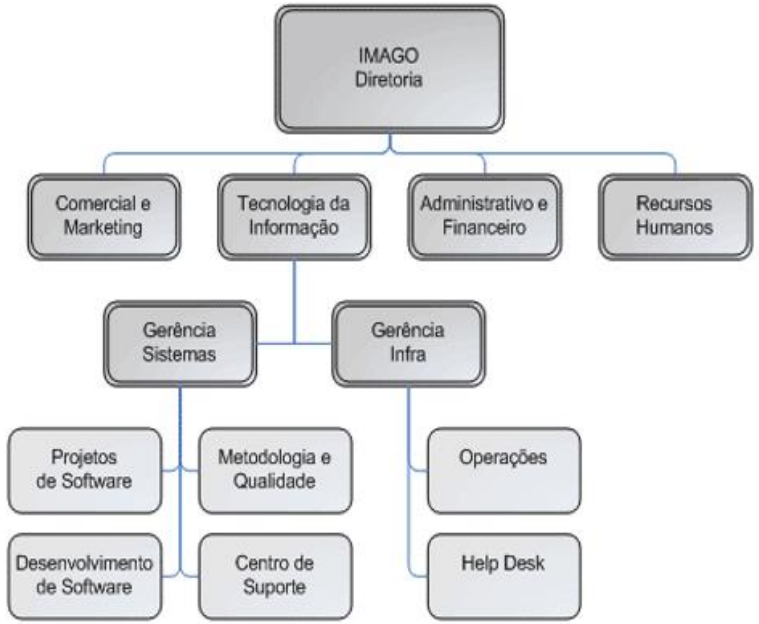
Prof. Marco André Abud Kappel

Aula 6 – Árvores de Decisão

Árvores de Decisão

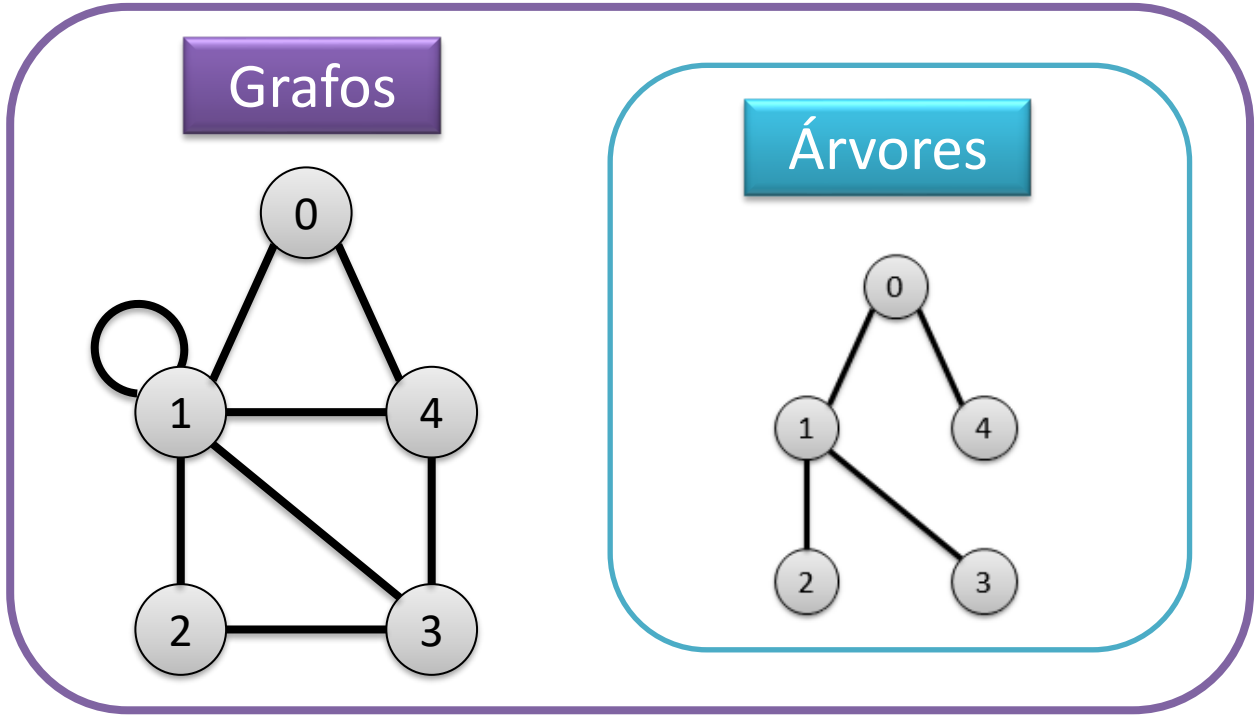
- **Introdução**
  - Uma árvore organiza um conjunto de acordo com uma **estrutura hierárquica**.





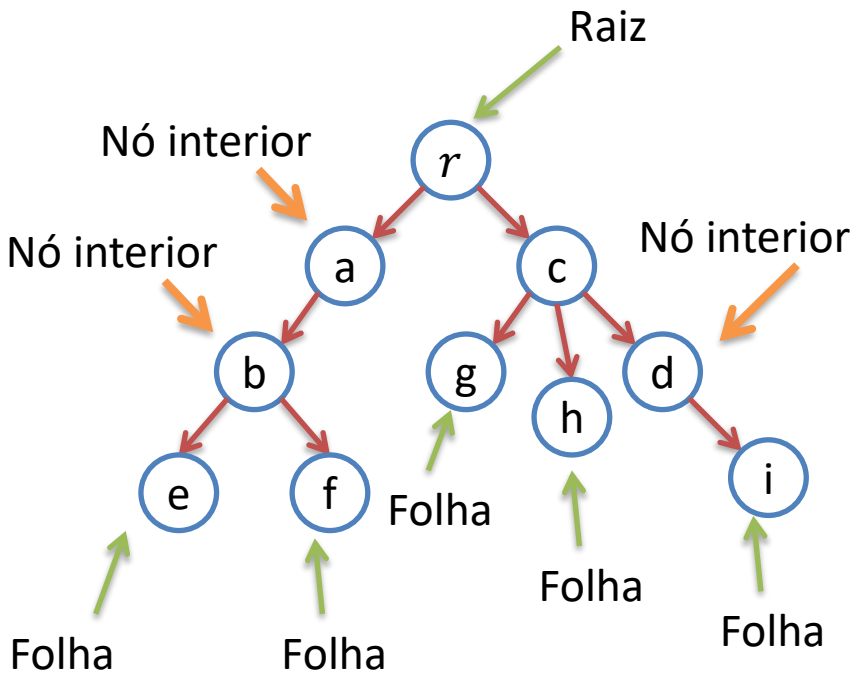
Árvores de Decisão

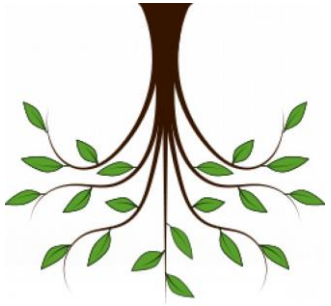
- **Introdução**
  - Árvores são **grafos conexos sem ciclos**.
  - Toda árvore é um grafo, mas nem todo grafo é uma árvore.



Árvores de Decisão

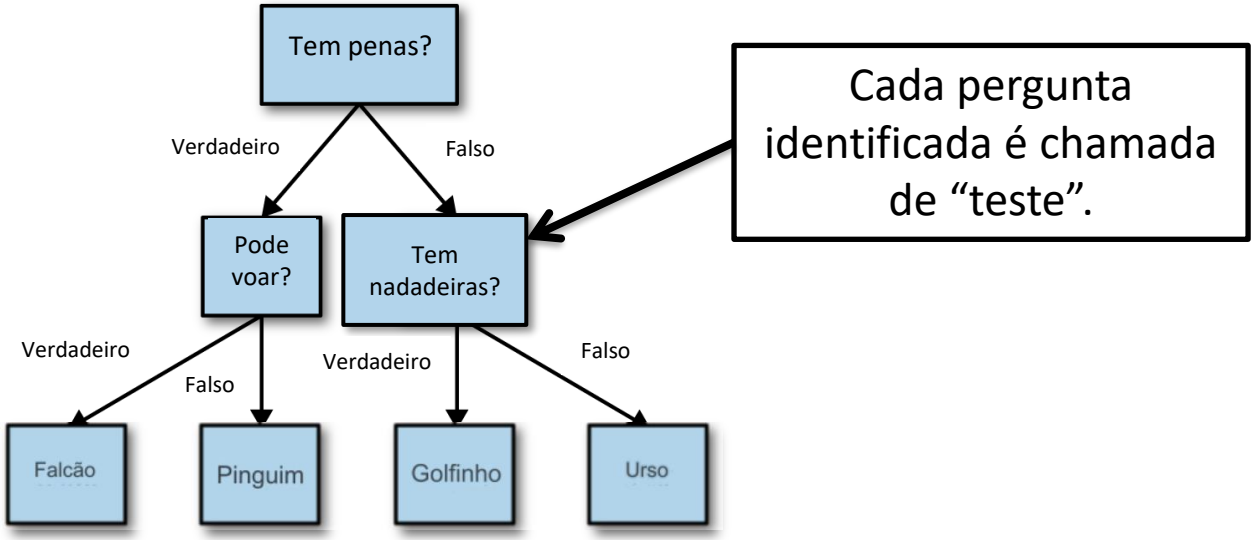
- **Introdução**
  - As árvores enraizadas possuem um nó especial r, chamado **nó raiz** da árvore.
  - Nós que não têm descendentes são chamados **folhas**.





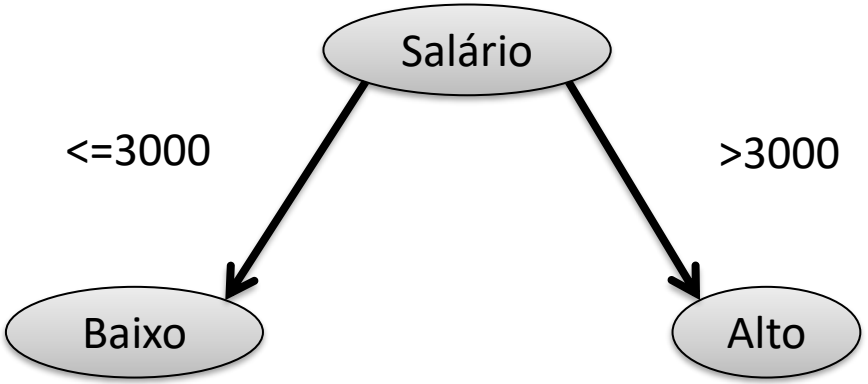
Árvores de Decisão

- **Descrição da técnica**
  - Trata-se de uma técnica muito utilizada tanto para tarefas de **regressão** quanto de **classificação**.
  - As árvores geradas aprendem uma **hierarquia** simples de if/elses, que levam a uma decisão.



Árvores de Decisão

- **Descrição da técnica**
  - Normalmente, os atributos de uma base de dados não aparecem na forma de **perguntas binárias**, como no exemplo anterior.
  - No lugar disso, os testes envolverão perguntas do tipo "Esta característica tem seu valor maior que o valor *x*?"

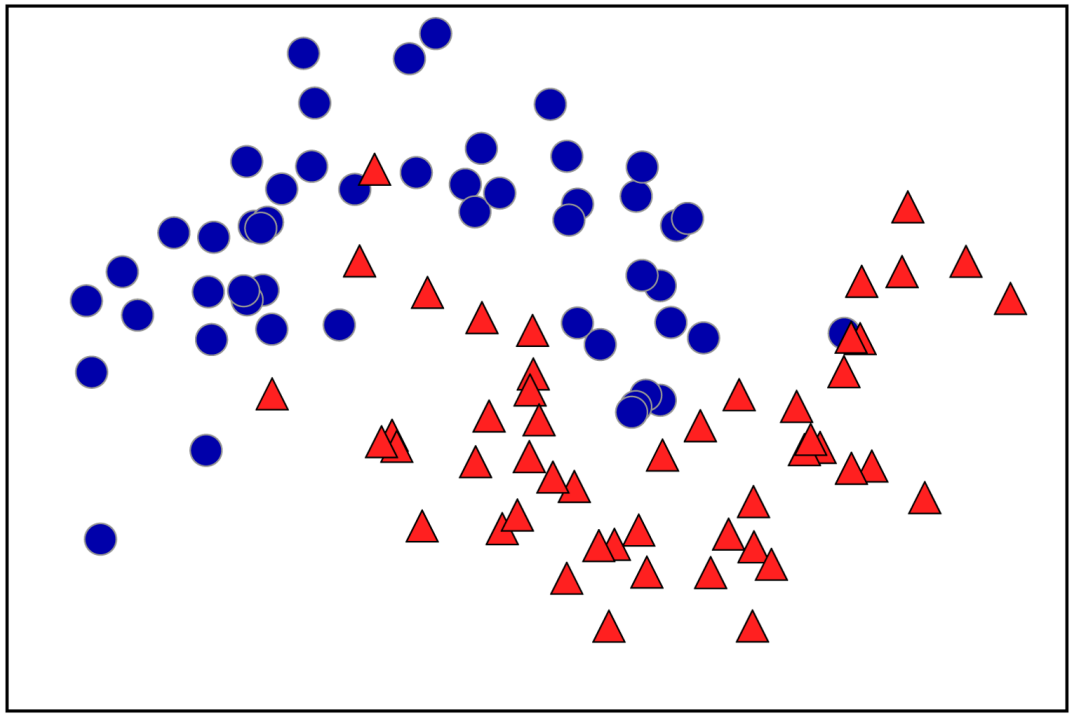


Árvores de Decisão

- **Descrição da técnica**
  - Como o algoritmo estuda o impacto **individualmente** de cada variável na classificação, normalmente não é necessário aplicar nenhuma **normalização** ou **padronização**.
  - Além disso, a visualização da árvore permite uma **análise profunda** da forma que uma classificação é feita.
  - Por isso, é uma boa técnica para ser adotada quando queremos descobrir o **quanto** uma característica está **afetando** a classificação.

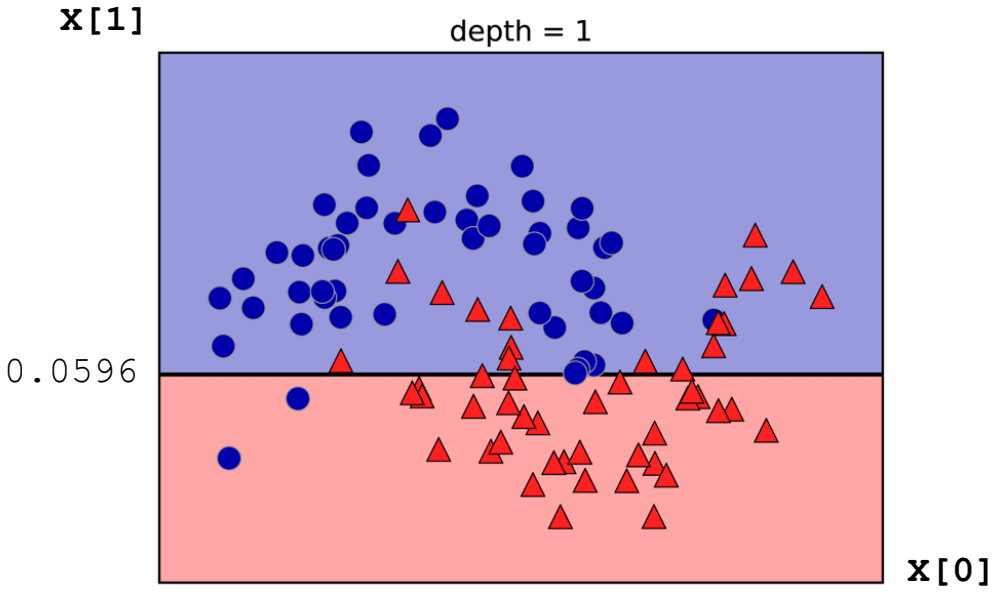
Árvores de Decisão

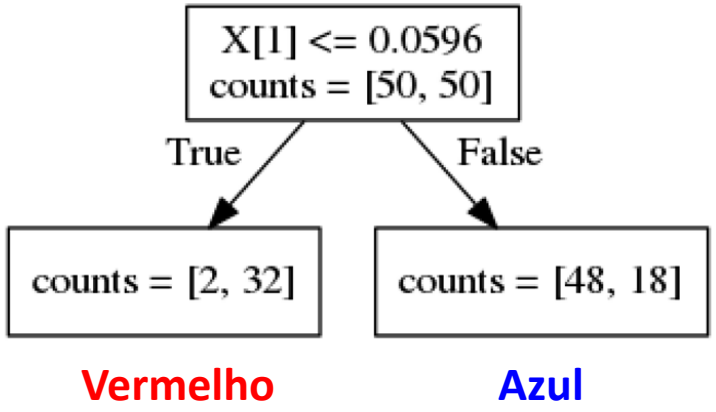
- **Descrição da técnica**
  - Suponha que queremos **classificar** os seguintes pontos em "**vermelhos**" ou "**azuis**".



Árvores de Decisão

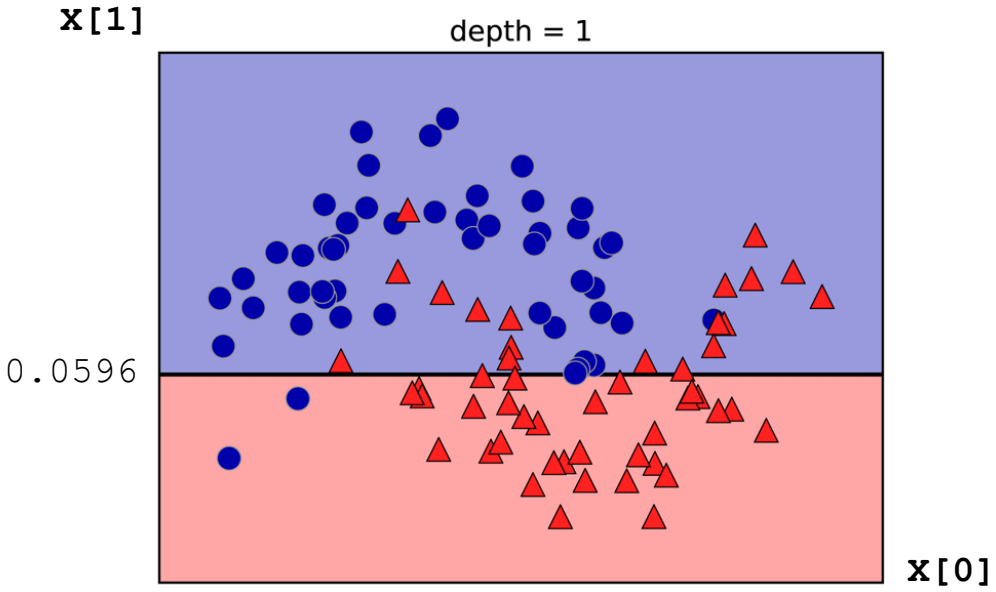
- **Descrição da técnica**
  - Para construir a árvore, o algoritmo investiga os **possíveis testes** e encontra aquele que é mais relevante para a variável em estudo.
  - Neste caso, o algoritmo definiu a **altura y** de 0,0596 como a melhor divisão.

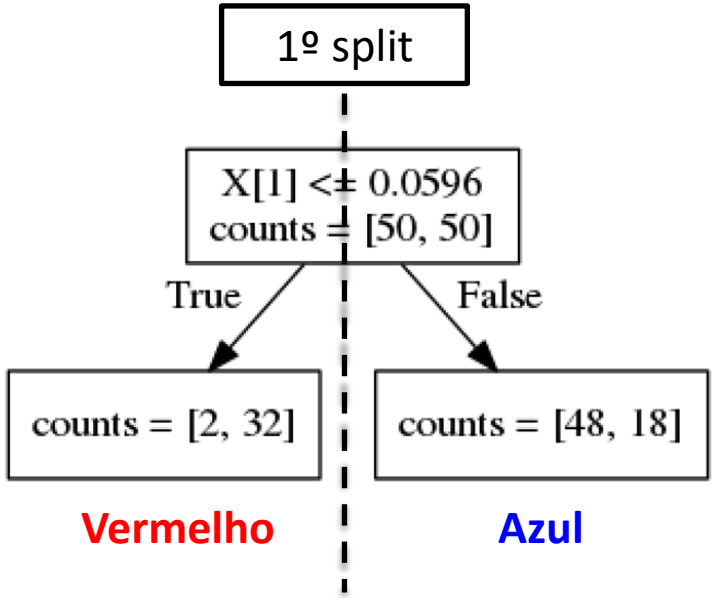




Árvores de Decisão

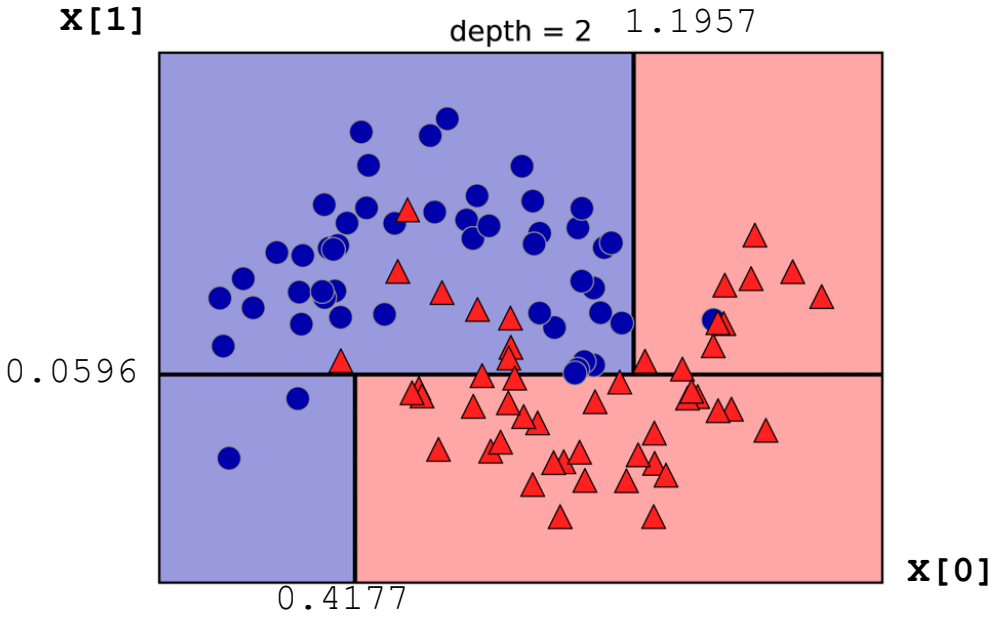
- **Descrição da técnica**
  - A **raiz** da árvore representa a **totalidade** de pontos.
  - Com esta divisão, a árvore classifica **corretamente** 32 pontos vermelhos e 48 pontos azuis.

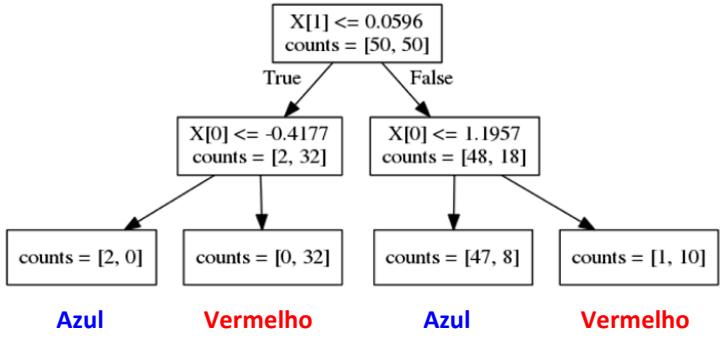




Árvores de Decisão

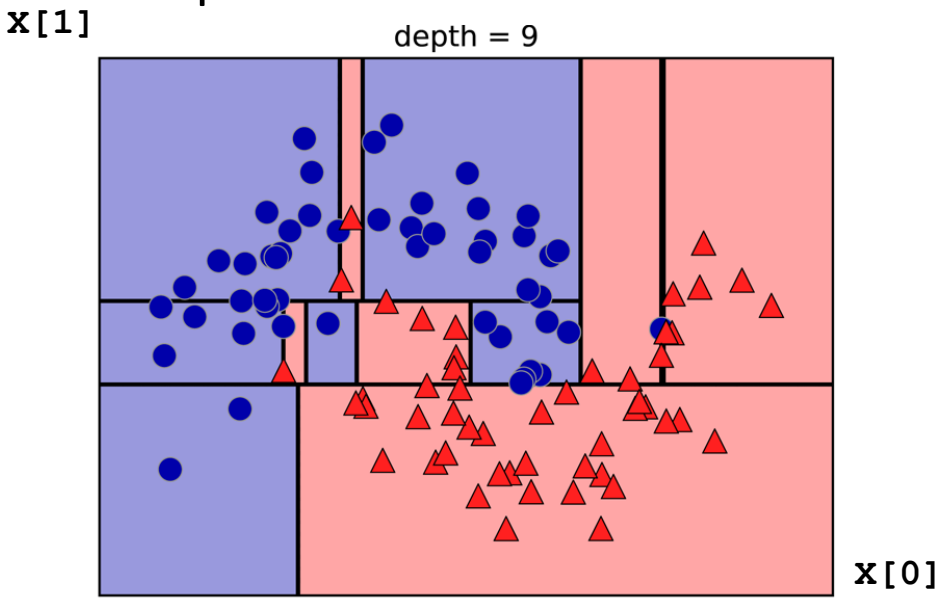
- **Descrição da técnica**
  - Podemos aplicar o mesmo processo **recursivamente**, encontrando as **melhores divisões** para as subárvores.
  - A classificação final é dada apenas pelas **folhas da árvore**.

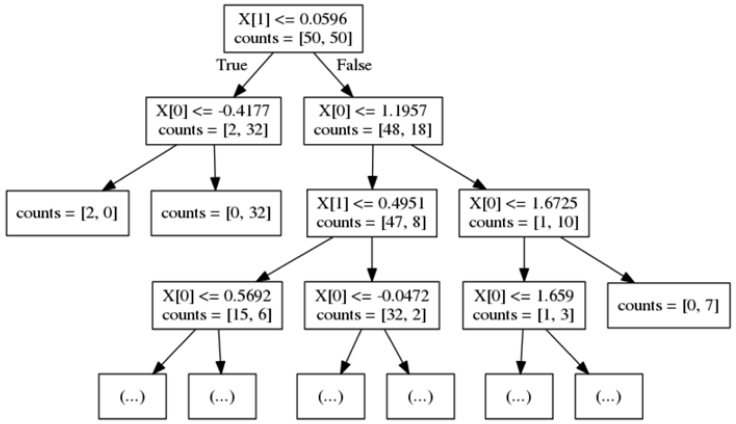




Árvores de Decisão

- **Descrição da técnica**
  - A previsão de um **novo registro** é baseado apenas em **percorrer a árvore** até chegar em uma folha.
  - É possível continuar o **particionamento** até que **cada folha** contenha apenas elementos de uma **única classe**.





Árvores de Decisão

- **Descrição da técnica**
  - A árvore pode crescer continuamente para tamanhos muito complexos.
  - Seguir realizando os splits até chegar nas folhas “puras” levaria a uma **taxa de acerto** de 100% na **base de treinamento**.
  - Porém, na **base de testes**, o resultado não seria tão bom.
  - O que isso indica?

Árvores de Decisão

- **Descrição da técnica**
  - A árvore pode crescer continuamente para tamanhos muito complexos.
  - Seguir realizando os splits até chegar nas folhas “puras” levaria a uma **taxa de acerto** de 100% na **base de treinamento**.
  - Porém, na **base de testes**, o resultado não seria tão bom.
  - O que isso indica?

Árvores de Decisão

- **Overfitting**
  - Para evitar o **crescimento desenfreado** de uma árvore, o que ocasionaria um “overfitting”, temos duas estratégia possíveis:

➢ Pré-poda

➢ Poda

Árvores de Decisão

- **Overfitting**
  - **Pré-poda:** o crescimento da árvore é limitado, por exemplo, indicando uma altura máxima para ela, ou indicando um número mínimo de registros pertencentes a um nó.

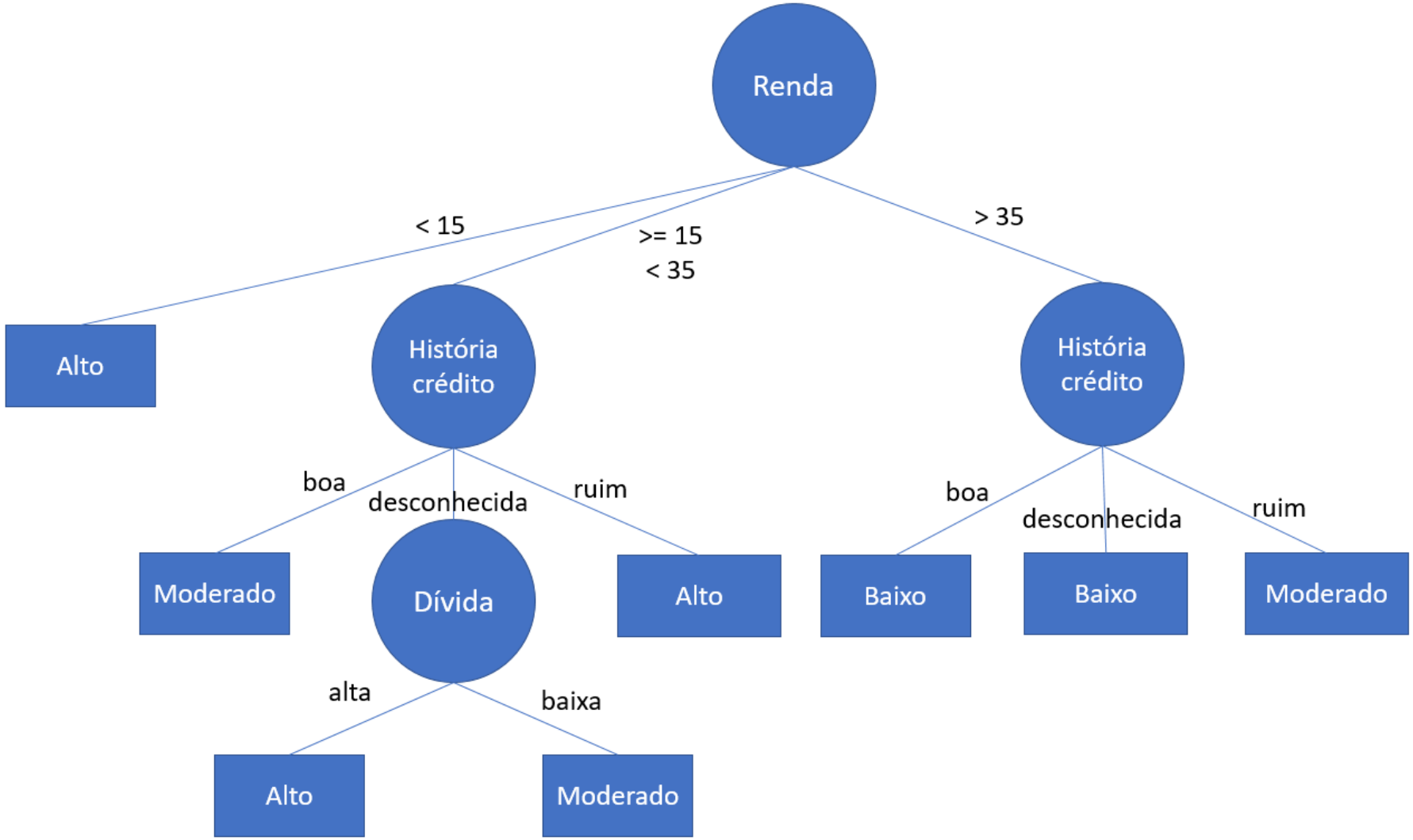
  - **Poda:** ramos da árvore são cortados após o fim de seu crescimento.

Árvores de Decisão

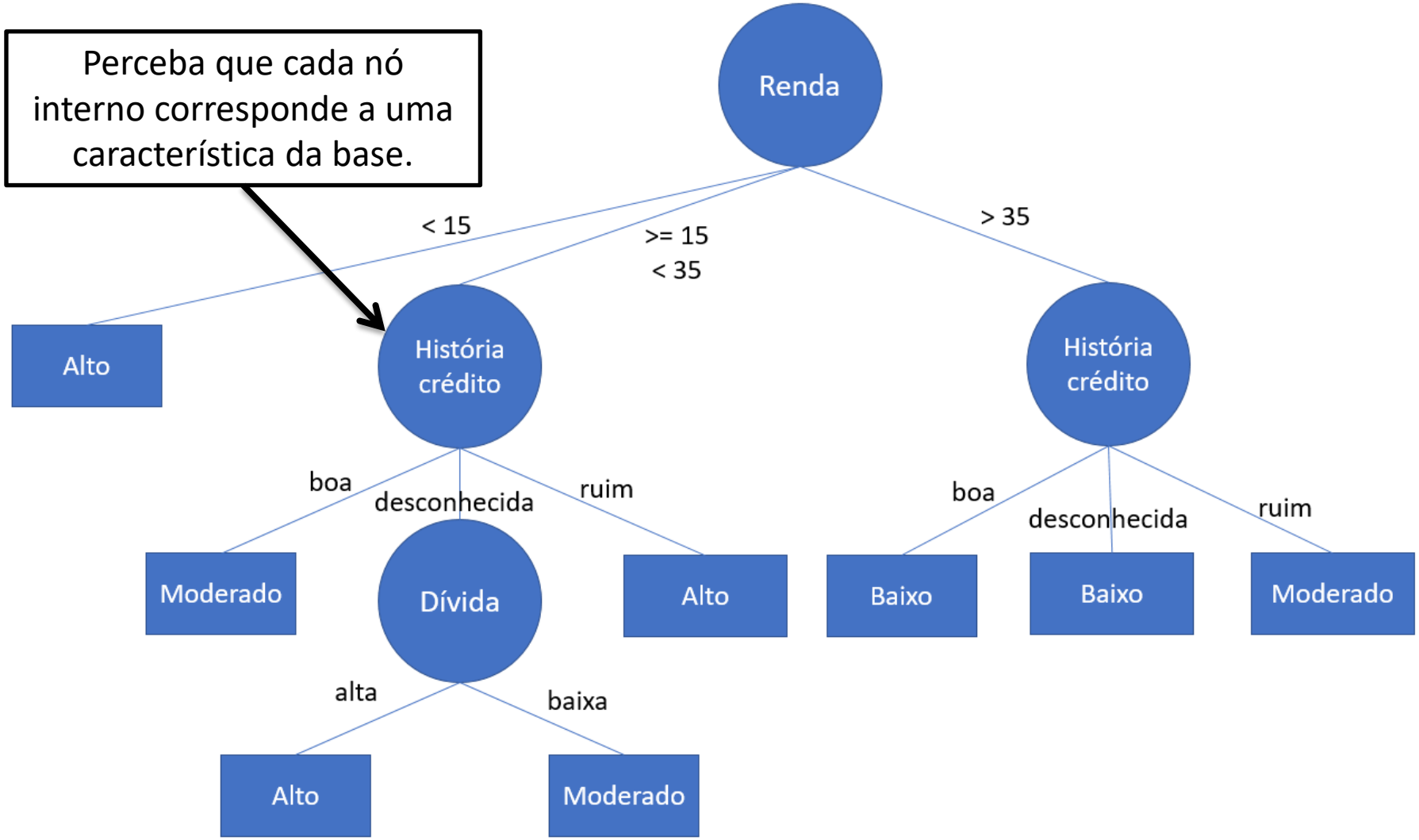
- **Exemplo**
  - Para mostrar como funciona o método Árvores de Decisão, vamos ver uma **aplicação** em uma base de dados simples.

História do crédito	Dívida	Garantias	Renda anual	Risco
Ruim	Alta	Nenhuma	< 15.000	Alto
Desconhecida	Alta	Nenhuma	>= 15.000 a <= 35.000	Alto
Desconhecida	Baixa	Nenhuma	>= 15.000 a <= 35.000	Moderado
Desconhecida	Baixa	Nenhuma	> 35.000	Alto
Desconhecida	Baixa	Nenhuma	> 35.000	Baixo
Desconhecida	Baixa	Adequada	> 35.000	Baixo
Ruim	Baixa	Nenhuma	< 15.000	Alto
Ruim	Baixa	Adequada	> 35.000	Moderado
Boa	Baixa	Nenhuma	> 35.000	Baixo
Boa	Alta	Adequada	> 35.000	Baixo
Boa	Alta	Nenhuma	< 15.000	Alto
Boa	Alta	Nenhuma	>= 15.000 a <= 35.000	Moderado
Boa	Alta	Nenhuma	> 35.0000	Baixo
Ruim	Alta	Nenhuma	>= 15.000 a <= 35.000	Alto

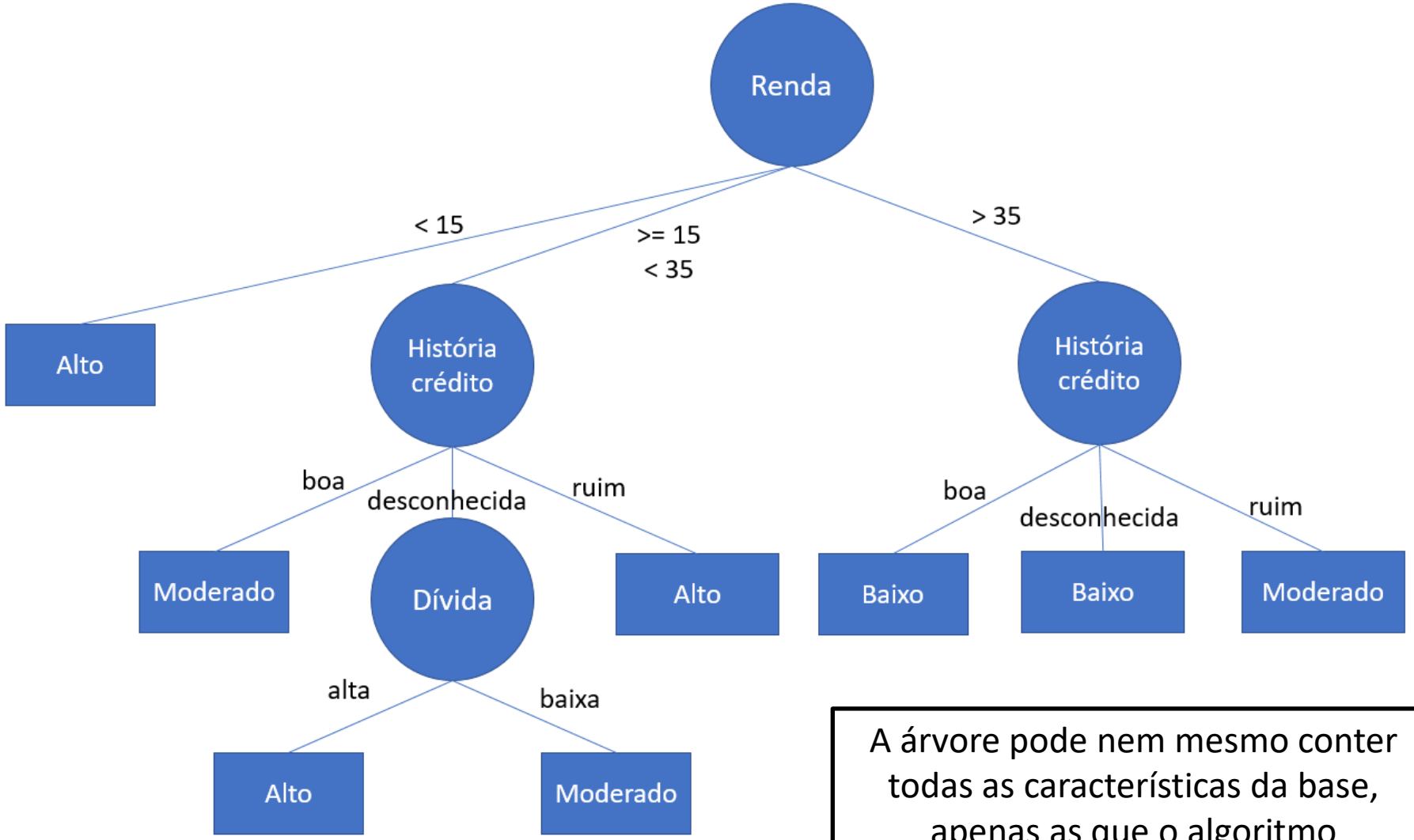
- **Exemplo**
  - No final da execução, teremos uma árvore da seguinte forma:



- **Exemplo**
  - No final da execução, teremos uma árvore da seguinte forma:

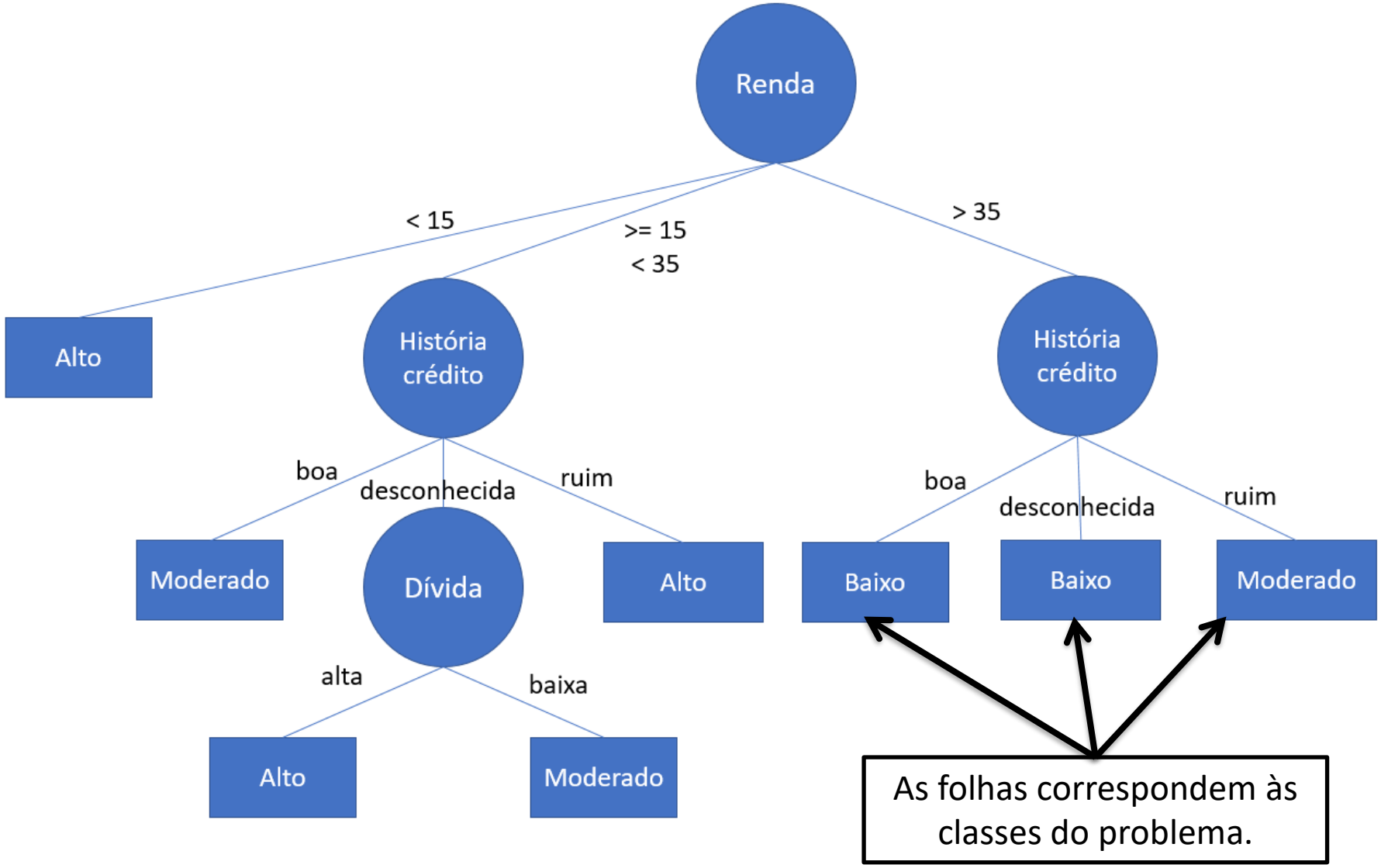


- **Exemplo**
  - No final da execução, teremos uma árvore da seguinte forma:



A árvore pode nem mesmo conter todas as características da base, apenas as que o algoritmo considerar mais importantes.

- **Exemplo**
  - No final da execução, teremos uma árvore da seguinte forma:

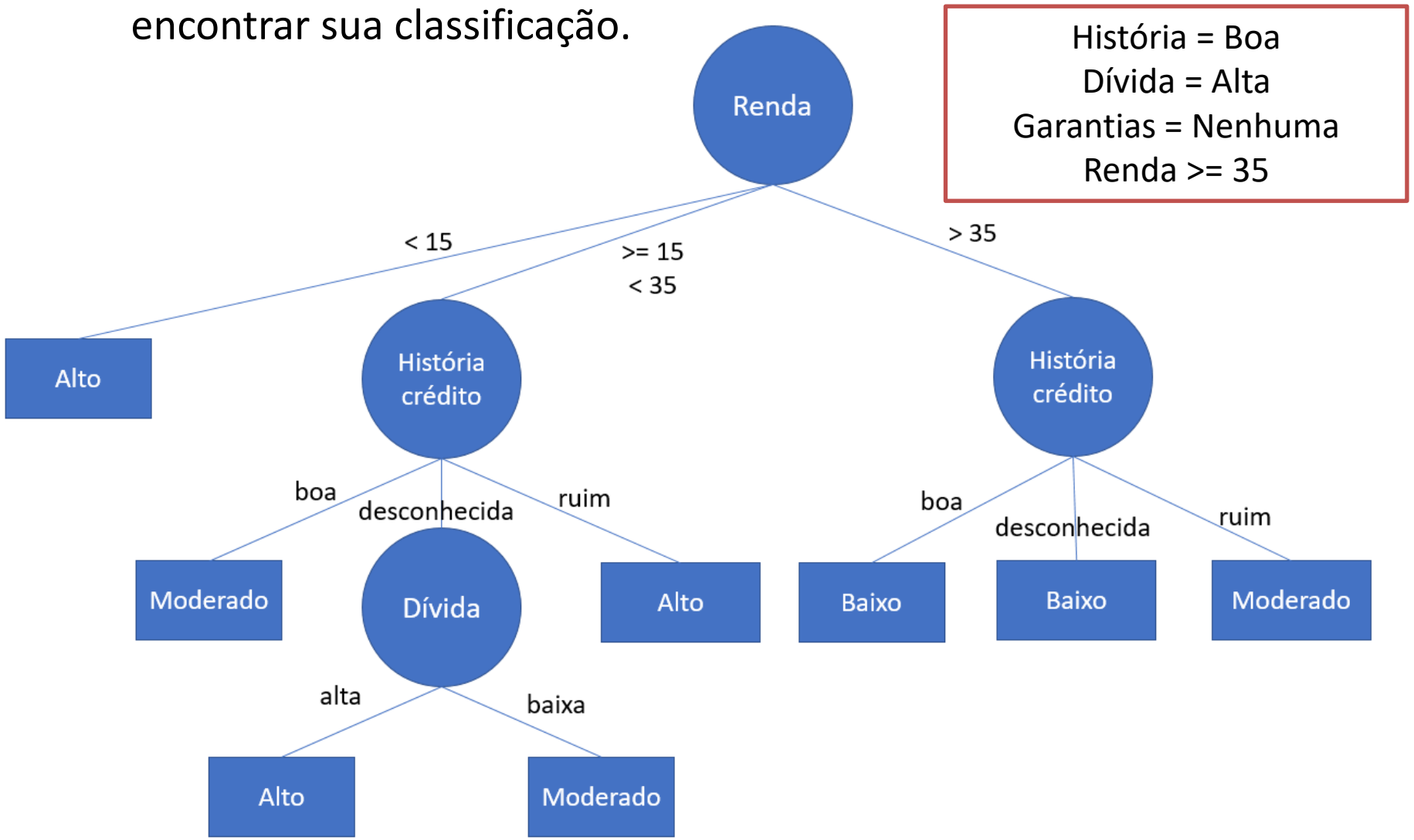


- **Exemplo**
  - Quando um **novo registro** chega: devemos percorrer a árvore para encontrar sua classificação.

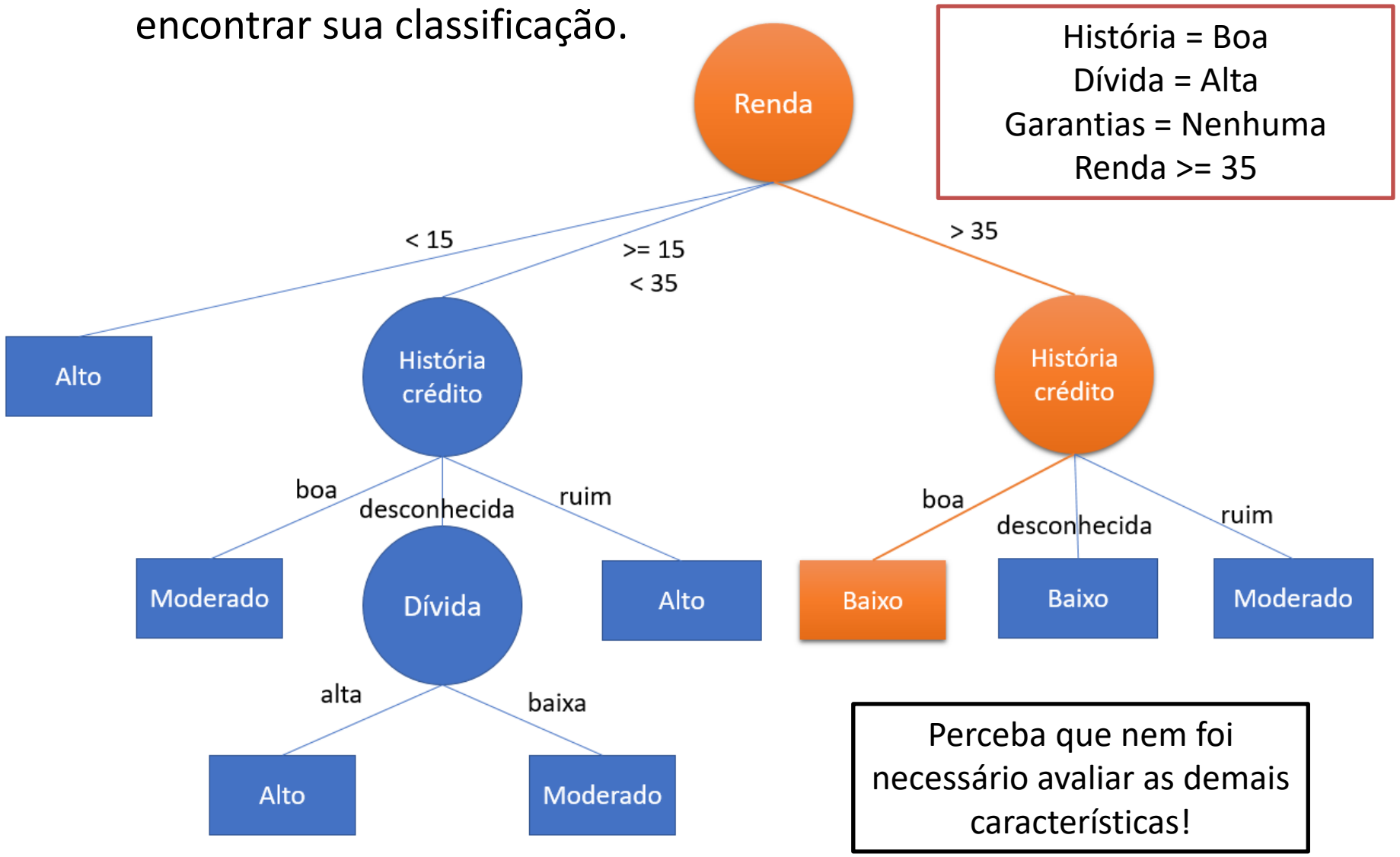
História = Boa
Dívida = Alta
Garantias = Nenhuma
Renda >= 35

História do crédito	Dívida	Garantias	Renda anual	Risco
Ruim	Alta	Nenhuma	< 15.000	Alto
Desconhecida	Alta	Nenhuma	>= 15.000 a <= 35.000	Alto
Desconhecida	Baixa	Nenhuma	>= 15.000 a <= 35.000	Moderado
Desconhecida	Baixa	Nenhuma	> 35.000	Alto
Desconhecida	Baixa	Nenhuma	> 35.000	Baixo
Desconhecida	Baixa	Adequada	> 35.000	Baixo
Ruim	Baixa	Nenhuma	< 15.000	Alto
Ruim	Baixa	Adequada	> 35.000	Moderado
Boa	Baixa	Nenhuma	> 35.000	Baixo
Boa	Alta	Adequada	> 35.000	Baixo
Boa	Alta	Nenhuma	< 15.000	Alto
Boa	Alta	Nenhuma	>= 15.000 a <= 35.000	Moderado
Boa	Alta	Nenhuma	> 35.0000	Baixo
Ruim	Alta	Nenhuma	>= 15.000 a <= 35.000	Alto

- **Exemplo**
  - Quando um **novo registro** chega: devemos percorrer a árvore para encontrar sua classificação.



- **Exemplo**
  - Quando um **novo registro** chega: devemos percorrer a árvore para encontrar sua classificação.

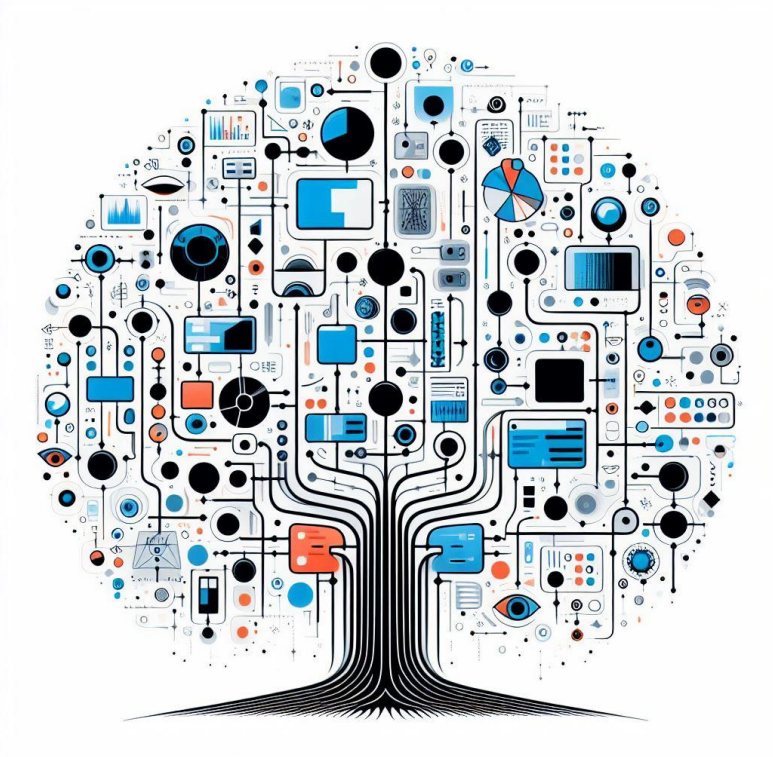


Árvores de Decisão

- **Construção da árvore**
  - Para construir a árvore, a abordagem mais comum do algoritmo faz uso de dois conceitos:

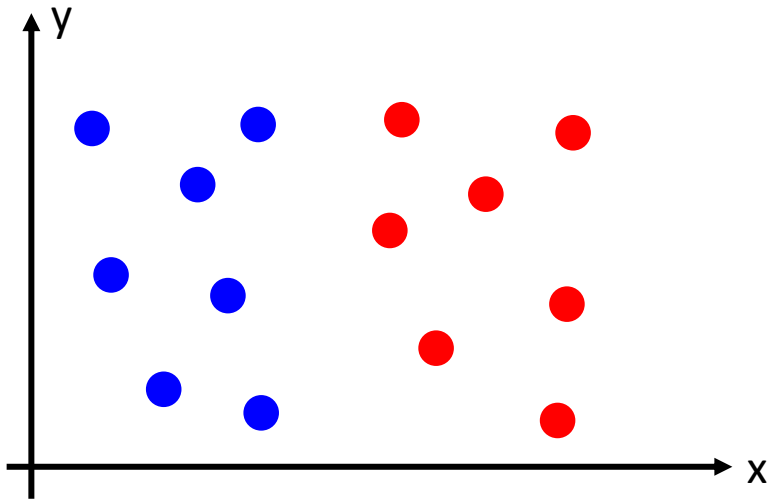
➢ **Entropia**

➢ **Ganho de Informação**



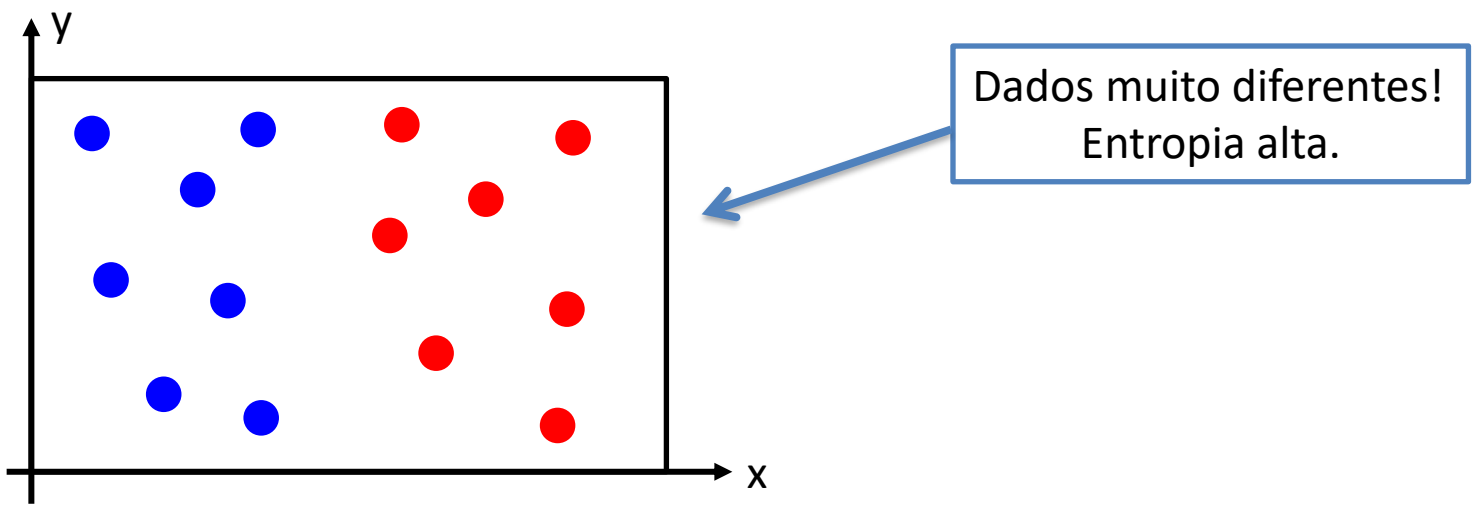
Árvores de Decisão

- **Construção da árvore**
  - **Entropia**: Mede o quanto os dados estão “organizados” ou “puros”. Se a entropia é baixa, a amostra está homogênea. Se a entropia é alta, a amostra está muito variada.
  - Exemplo:



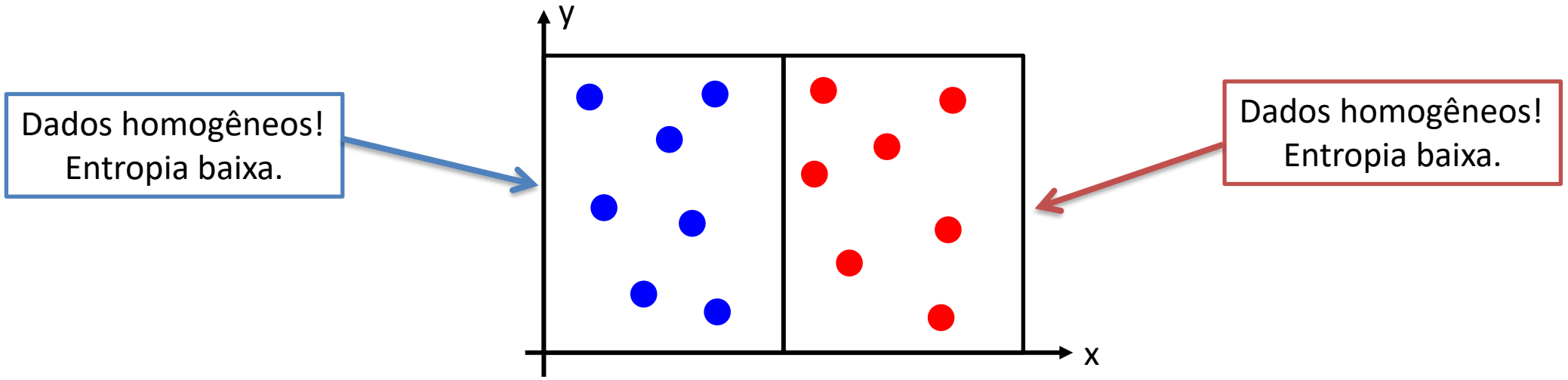
Árvores de Decisão

- **Construção da árvore**
  - **Entropia**: Mede o quanto os dados estão “organizados” ou “puros”. Se a entropia é baixa, a amostra está homogênea. Se a entropia é alta, a amostra está muito variada.
  - Exemplo:



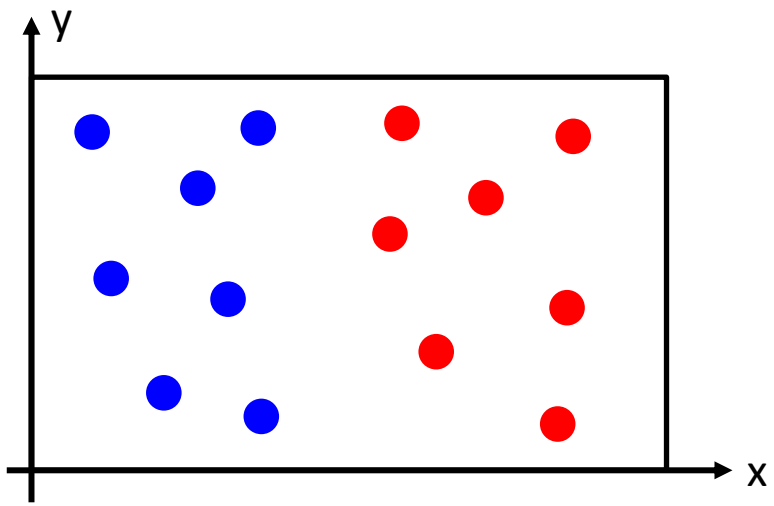
Árvores de Decisão

- **Construção da árvore**
  - **Entropia**: Mede o quanto os dados estão “organizados” ou “puros”. Se a entropia é baixa, a amostra está homogênea. Se a entropia é alta, a amostra está muito variada.
  - Exemplo:



Árvores de Decisão

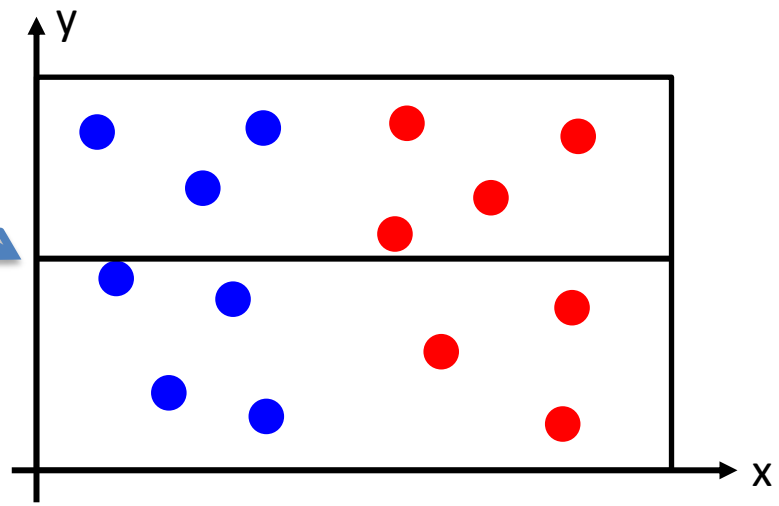
- **Construção da árvore**
  - **Ganho de informação**: Mede o quanto a entropia **diminuiu** quando a base de dados é dividida em um atributo.
  - Exemplo:



Árvores de Decisão

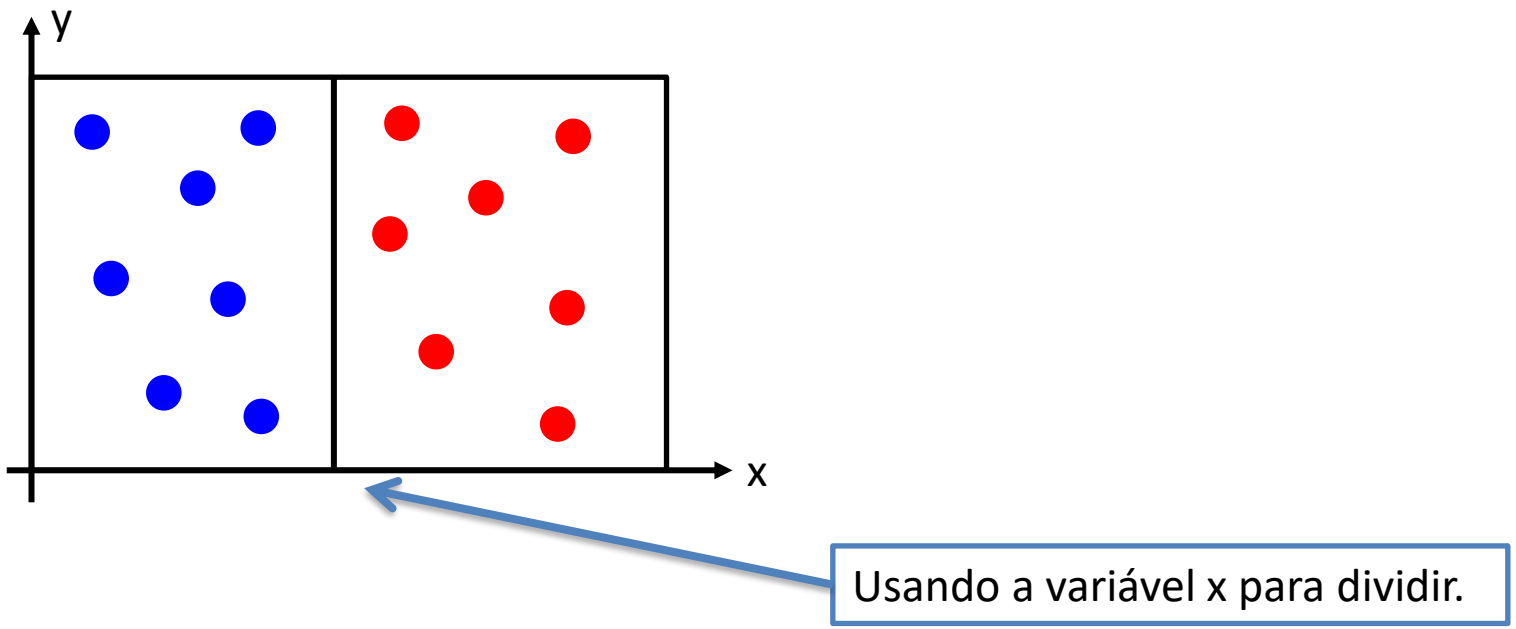
- **Construção da árvore**
  - **Ganho de informação**: Mede o quanto a entropia **diminuiu** quando a base de dados é dividida em um atributo.
  - Exemplo:

Usando a variável y para dividir.



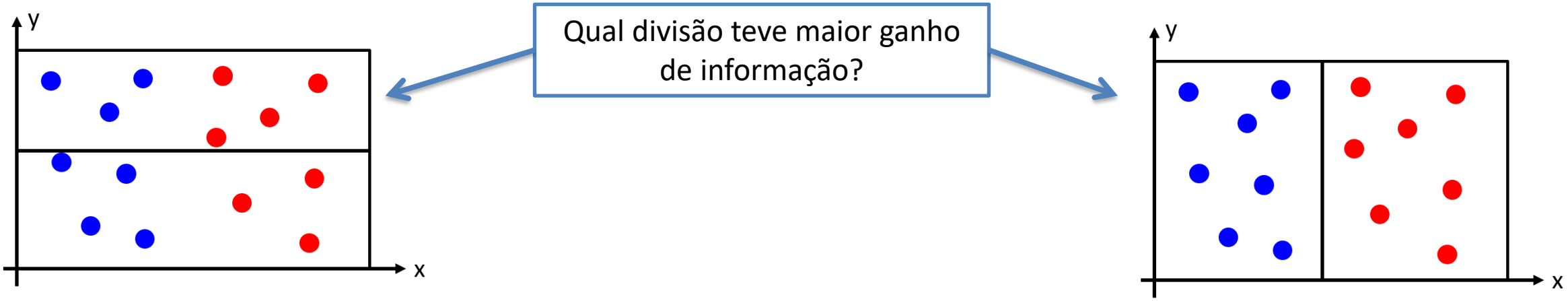
Árvores de Decisão

- **Construção da árvore**
  - **Ganho de informação**: Mede o quanto a entropia **diminuiu** quando a base de dados é dividida em um atributo.
  - Exemplo:



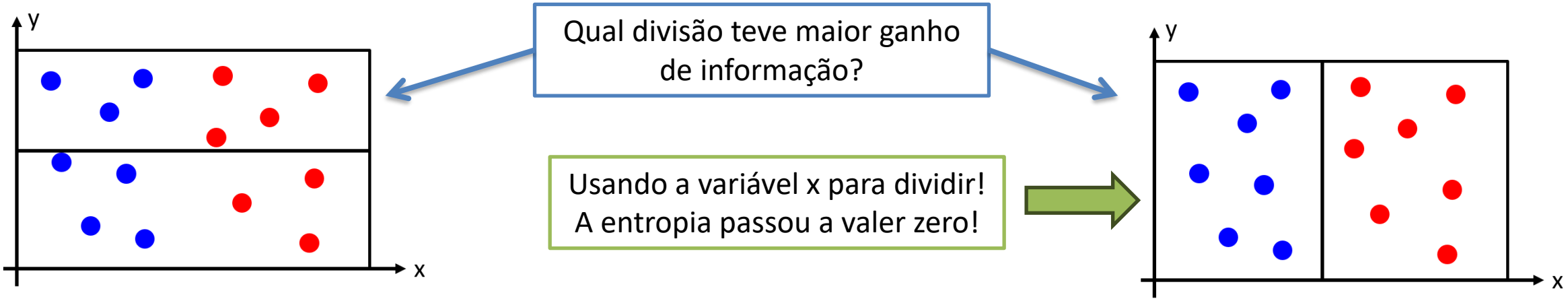
Árvores de Decisão

- **Construção da árvore**
  - **Ganho de informação**: Mede o quanto a entropia **diminuiu** quando a base de dados é dividida em um atributo.
  - Exemplo:



Árvores de Decisão

- **Construção da árvore**
  - **Ganho de informação**: Mede o quanto a entropia **diminuiu** quando a base de dados é dividida em um atributo.
  - Exemplo:



Árvores de Decisão

- **Construção da árvore**
  - Construir a árvore é, basicamente, descobrir quais **atributos são os mais relevantes** para a tomada de decisão.
  - O **cálculo** destas duas quantidades é dado pelas seguintes fórmulas:

➢ **Entropia**: $$\mathrm{E(S)} = \sum_{i=1}^{c} -p_i \log_2 p_i$$

S é um conjunto de dados, p_i é a frequência de cada classe nesse conjunto.

Árvores de Decisão

- **Construção da árvore**
  - A divisão dos dados será realizada no atributo que promover o maior ganho de informação.

➢ **Ganho de informação**: $$\mathrm{G(S,A)} = \mathrm{E(S)} - \sum_{v \epsilon A} \frac{|S_v|}{|S|} E(S_v)$$

A é um atributo da base, S_v é o subconjunto de S com a característica A presente.

- **Exemplo**
  - Vamos calcular a entropia da base de dados:

Risco
Alto
Alto
Moderado
Alto
Baixo
Baixo
Alto
Moderado
Baixo
Baixo
Alto
Moderado
Baixo
Alto

$$\mathrm{E(S)} = \sum_{i=1}^{c} -p_i \log_2 p_i$$

Alto = 6/14

Moderado = 3/14

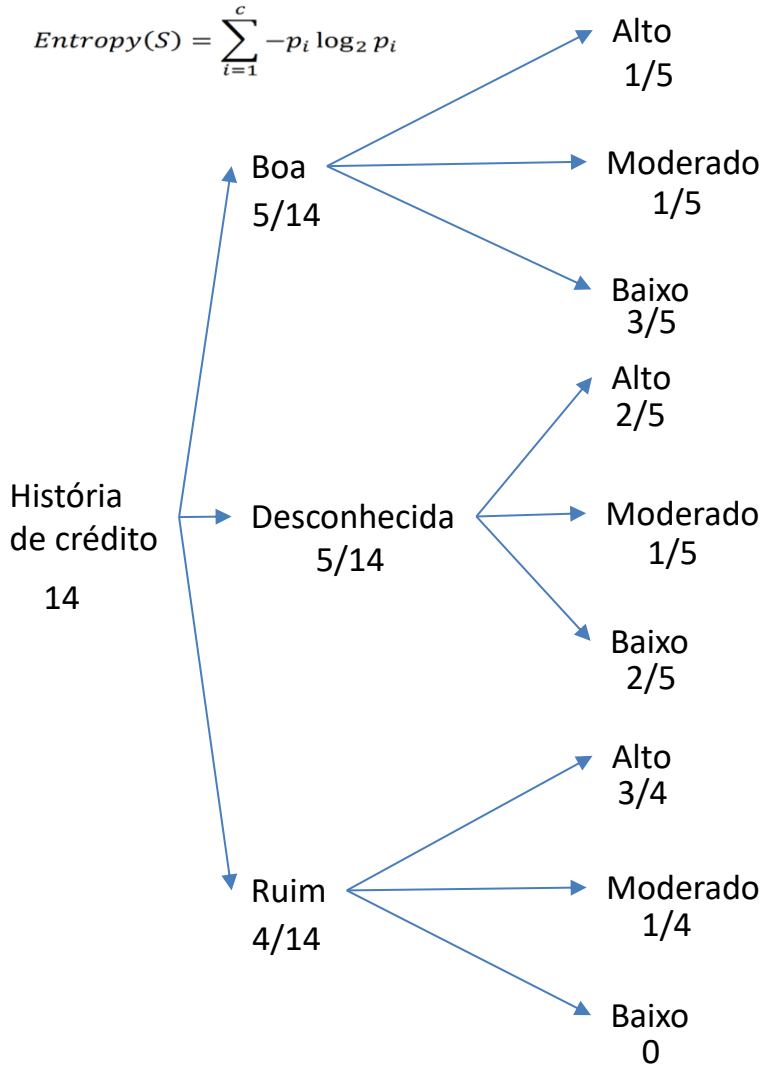
Baixo = 5/14

E(s) = -6/14 * log(6/14; 2) – 3/14 * log(3/14; 2) – 5/14 * log(5/14; 2) = **1,53**

– Agora, precisamos calcular qual atributo possui o maior ganho de informação e, assim, se tornar a raiz da árvore.

História do crédito	Risco
Ruim	Alto
Desconhecida	Alto
Desconhecida	Moderado
Desconhecida	Alto
Desconhecida	Baixo
Desconhecida	Baixo
Ruim	Alto
Ruim	Moderado
Boa	Baixo
Boa	Baixo
Boa	Alto
Boa	Moderado
Boa	Baixo
Ruim	Alto

$$Entropy(S) = \sum_{i=1}^{c} -p_i \log_2 p_i$$



$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

E(s) = -1/5 * log(1/5; 2) – 1/5 * log(1/5; 2) – 3/5 * log(3/5; 2) = **1,37**

E(s) = -2/5 * log(2/5; 2) – 1/5 * log(1/5; 2) – 2/5 * log(2/5; 2) = **1,52**

Ganho(História) = 1,53 – (5/14 * 1,37) – (5/14 * 1,52) – (4/14 * 0,81) = 0,26

E(s) = -3/4 * log(3/4; 2) – 1/4 * log(1/4; 2) – 0 * log(0; 2) = **0,81**

– O próximo passo é fazer o mesmo cálculo para as outras características:

Dívida	Risco
Alta	Alto
Alta	Alto
Baixa	Moderado
Baixa	Alto
Baixa	Baixo
Baixa	Baixo
Baixa	Alto
Baixa	Moderado
Baixa	Baixo
Alta	Baixo
Alta	Alto
Alta	Moderado
Alta	Baixo
Alta	Alto

$$Entropy(S) = \sum_{i=1}^{c} -p_i \log_2 p_i$$

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Dívida
14

Alta
7/14

Alto
4/7

Moderado
1/7

Baixo
2/7

E(s) = -4/7 * log(4/7; 2) – 1/7 * log(1/7; 2) – 2/7 * log(2/7; 2) = **1,38**

Ganho(Dívida) = 1,53 – (7/14 * 1,38) – (7/14 * 1,56) = 0,06

Baixa
7/14

Alto
2/7

Moderado
2/7

Baixo
3/7

E(s) = -2/7 * log(2/7; 2) – 2/7 * log(2/7; 2) – 3/7 * log(3/7; 2) = **1,56**

– O próximo passo é fazer o mesmo cálculo para as outras características:

Garantias	Risco
Nenhuma	Alto
Nenhuma	Alto
Nenhuma	Moderado
Nenhuma	Alto
Nenhuma	Baixo
Adequada	Baixo
Nenhuma	Alto
Adequada	Moderado
Nenhuma	Baixo
Adequada	Baixo
Nenhuma	Alto
Nenhuma	Moderado
Nenhuma	Baixo
Nenhuma	Alto

$$Entropy(S) = \sum_{i=1}^{c} -p_i \log_2 p_i$$

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Garantias
14

Nenhuma
11/14

Alto
6/11

Moderado
2/11

Baixo
3/11

E(s) = -6/11 * log(6/11; 2) – 2/11 * log(2/11; 2) – 3/11 * log(3/11; 2) = **1,44**

Ganho(Garantias) = 1,53 – (11/14 * 1,44) – (3/14 * 0,92) = 0,20

Adequada
3/14

Alto
0

Moderado
1/3

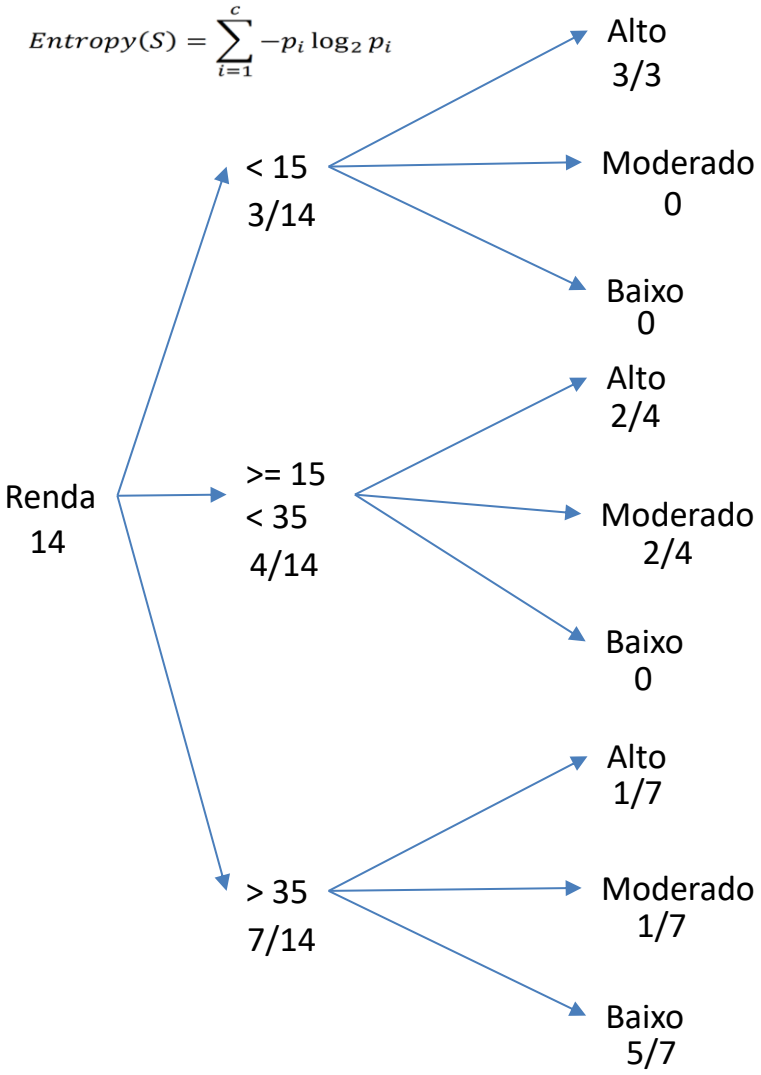
Baixo
2/3

E(s) = -0 * log(0; 2) – 1/3 * log(1/3; 2) – 2/3 * log(2/3; 2) = **0,92**

– O próximo passo é fazer o mesmo cálculo para as outras características:

Renda anual	Risco
< 15.000	Alto
>= 15.000 a <= 35.000	Alto
>= 15.000 a <= 35.000	Moderado
> 35.000	Alto
> 35.000	Baixo
> 35.000	Baixo
< 15.000	Alto
> 35.000	Moderado
> 35.000	Baixo
> 35.000	Baixo
< 15.000	Alto
>= 15.000 a <= 35.000	Moderado
> 35.0000	Baixo
>= 15.000 a <= 35.000	Alto

$$Entropy(S) = \sum_{i=1}^{c} -p_i \log_2 p_i$$



$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

E(s) = -3/3 * log(3/3; 2) – 0 * log(0; 2) – 0 * log(0 2) = **0,00**

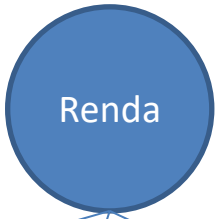
E(s) = -2/4 * log(2/4; 2) – 2/4 * log(2/4; 2) – 0 * log(0; 2) = **1,00**

Ganho(Renda) = 1,53 – (3/14 * 0,00) – (4/14 * 1,00) – (7/14 * 1,15) = 0,66

E(s) = -1/7 * log(1/7; 2) – 1/7 * log(1/7; 2) – 5/7 * log(5/7; 2) = **1,15**

- Neste caso, a **Renda** foi a característica **mais importante**.
- Assim, ela se torna a **raiz** da árvore de decisão.
- O **mesmo procedimento** deve ser aplicado, agora, nas **três subárvores** restantes.

História de crédito = 0,26
Dívida = 0,06
Garantias = 0,20
Renda = 0,66



< 15

História do crédito	Dívida	Garantias	Renda anual	Risco
Ruim	Alta	Nenhuma	< 15.000	Alto
Ruim	Baixa	Nenhuma	< 15.000	Alto
Boa	Alta	Nenhuma	< 15.000	Alto

>= 15
< 35

História do crédito	Dívida	Garantias	Renda anual	Risco
Desconhecida	Alta	Nenhuma	>= 15.000 a <= 35.000	Alto
Desconhecida	Baixa	Nenhuma	>= 15.000 a <= 35.000	Moderado
Boa	Alta	Nenhuma	>= 15.000 a <= 35.000	Moderado
Ruim	Alta	Nenhuma	>= 15.000 a <= 35.000	Alto

> 35

História do crédito	Dívida	Garantias	Renda anual	Risco
Desconhecida	Baixa	Nenhuma	> 35.000	Alto
Desconhecida	Baixa	Nenhuma	> 35.000	Baixo
Desconhecida	Baixa	Adequada	> 35.000	Baixo
Ruim	Baixa	Adequada	> 35.000	Moderado
Boa	Baixa	Nenhuma	> 35.000	Baixo
Boa	Alta	Adequada	> 35.000	Baixo
Boa	Alta	Nenhuma	> 35.0000	Baixo

- **Construindo o script**

  – Primeiramente, o script de pré-processamento deve ser executado, selecionando quais as transformações precisam ser feitas na base.

Spyder (Python 3.12)

Arquivo Editar Pesquisar Código Executar Depurar Consoles Projetos Ferramentas Ver Ajuda

E:\PythonWorkspace\2025.1\aula4

...tual\2025-1_Técnicas de Aprendizado de Máquina (reformulação)\aula 3\Programas\pre-processamento.py

pre-processamento.py × credit_data.csv ×

```python
# -*- coding: utf-8 -*-
"""
Created on Mon Mar 31 18:32:59 2025

@author: marco
"""

import pandas as pd
base = pd.read_csv('credit_data.csv')
resumo = base.describe()


# ==========================================================================
#                        Tratando valores inválidos
# ==========================================================================

# Verifica idades inválidas
base.loc[base['Age'] < 0]

# Apagar a coluna
# base.drop('Age', axis=1, inplace=True)

# Apagar apenas os registros com problema
# base.drop(base[base['Age'] < 0].index, inplace=True)

# Preencher os valores com a média
idade_media = base['Age'][base['Age'] > 0].mean().round()
base.loc[base['Age'] < 0, 'Age'] = idade_media

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# Preencher os valores com o mais frequente
saving_maioria = base['Saving accounts'][pd.notnull(base['Saving accounts'])]
```

Nome	Tipo	Tamanho	Valor
base	DataFrame	(1000, 11)	Column names: Id, Age, …
checking_maioria	str	6	little
classe	DataFrame	(600, 1)	Column names: Risk
classe_teste	DataFrame	(150, 1)	Column names: Risk
classe_treinamento	DataFrame	(450, 1)	Column names: Risk
cols_classe	list	1	['Risk']
cols_previsores	Index	(18,)	Index: 18 entries, Age …

Ajuda Explorador de variáveis Figuras Arquivos

Console 1/A ×

```
Python 3.12.4 | packaged by Anaconda, Inc. | (main, Jun 18 2024,
15:03:56) [MSC v.1929 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 8.25.0 -- An enhanced Interactive Python.

In [1]:
   ...: """
   ...: Created on Mon Mar 31 18:32:59 2025
   ...:
   ...: @author: marco
   ...: """
   ...:
   ...: import pandas as pd
   ...: base = pd.read_csv('credit_data.csv')
```

Console IPython Histórico

conda: base (Python 3.12.4) Completions: conda(base) LSP: Python Line 3, Col 36 UTF-8 CRLF RW Mem 75%

- **Construindo o script**
  - Para construir as Árvores de Decisão, crie um novo arquivo chamado "arvore_de_decisao.py".
  - Nele, insira o seguinte código:

```
import pandas as pd

################## Classificação com Árvores de Decisão ##################

# Geração da árvore
from sklearn.tree import DecisionTreeClassifier
classificador = DecisionTreeClassifier(max_depth=?
                                       criterion='entropy',
                                       random_state=0)

# Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

# Teste
previsoes = classificador.predict(previsores_teste)

# Análise dos resultados (porcentagem de acertos e MATRIZ DE CONFUSÃO)
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste, previsoes)
matriz_teste = confusion_matrix(classe_teste, previsoes)

# Exportando a árvore para fazer figura
from sklearn.tree import export_graphviz
export_graphviz(classificador,out_file="tree.dot",class_names=["bad","good"],
                feature_names=cols_previsores, impurity=False, filled=True)

# Visualizando a árvore
import graphviz
with open("tree.dot") as f:
    dot_graph = f.read()
display(graphviz.Source(dot_graph))
```

- **Construindo o script**
  - Para construir as Árvores de Decisão, crie um novo arquivo chamado "arvore_de_decisao.py".
  - Nele, insira o seguinte código:

```
import pandas as pd

################## Classificação com Árvores de Decisão ##########

# Geração da árvore
from sklearn.tree import DecisionTreeClassifier
classificador = DecisionTreeClassifier(max_depth=?
                                       criterion='entropy',
                                       random_state=0)

# Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

classificador.predict(previsores_teste)

s resultados (porcentagem de acertos e MATRIZ DE CON
.metrics import confusion_matrix, accuracy_score
te = accuracy_score(classe_teste, previsoes)
= confusion_matrix(classe_teste, previsoes)

# Exportando a árvore para fazer figura
from sklearn.tree import export_graphviz
export_graphviz(classificador,out_file="tree.dot",class_names=["bad","good"],
                feature_names=cols_previsores, impurity=False, filled=True)

# Visualizando a árvore
import graphviz
with open("tree.dot") as f:
    dot_graph = f.read()
display(graphviz.Source(dot_graph))
```

O valor de profundidade máxima da árvore precisa ser definido.

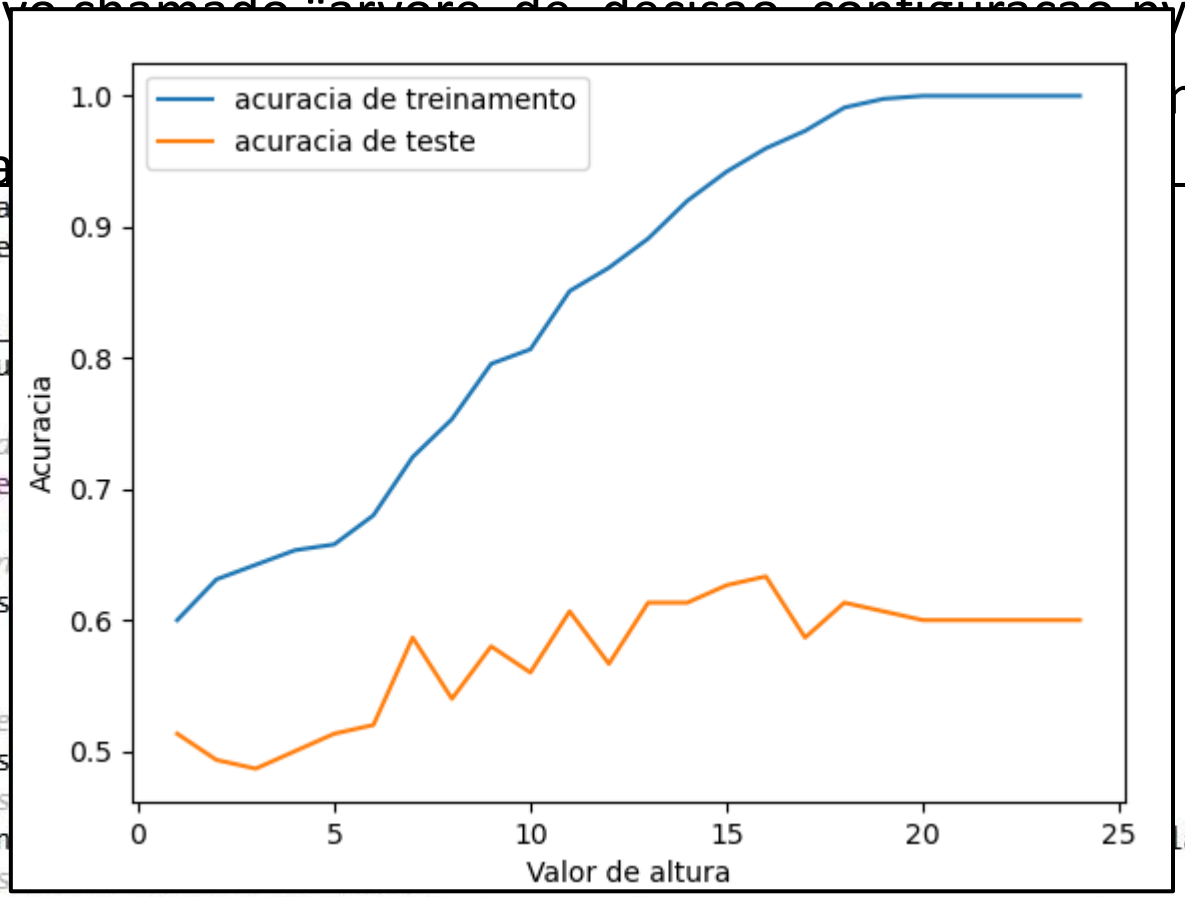
Nesta linha, definimos os parâmetros de configuração do método.

Indica que a métrica utilizada será a entropia.

- **Construindo o script**
  - Para descobrir a melhor configuração do algoritmo, crie um novo arquivo chamado “arvore_de_decisao_configuracao.py”.
  - Este script servirá para testarmos diferentes configurações e avaliarmos qual seria a ideal para este problema.

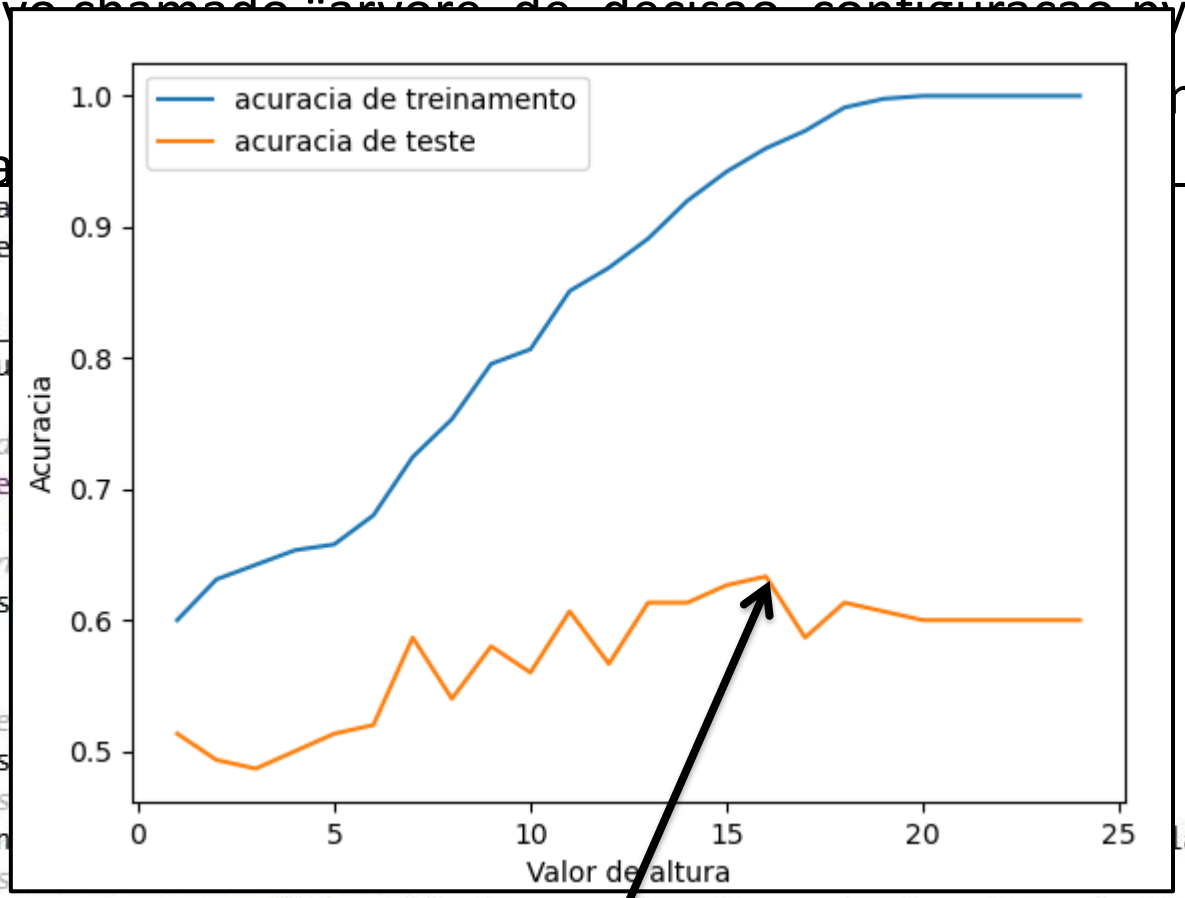
```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier

training_accuracy = []
test_accuracy = []

# tentando diferentes valores de K: de 1 a 20
h = range(1,21)
for i in h:
    # Construir o modelo
    classificador = DecisionTreeClassifier(max_depth=i,
                                           criterion='entropy',
                                           random_state=0)
    # Treinamento
    classificador.fit(previsores_treinamento, classe_treinamento)
    # resultado na base de treinamento
    training_accuracy.append(classificador.score(previsores_treinamento, classe_treinamento))
    # resultado na base de teste
    test_accuracy.append(classificador.score(previsores_teste, classe_teste))

plt.plot(h, training_accuracy, label="training accuracy")
plt.plot(h, test_accuracy, label="test accuracy")
plt.ylabel("Accuracy")
plt.xlabel("K")
plt.legend()
```

- **Construindo o script**
  - Para descobrir a melhor configuração do algoritmo, crie um novo arqui [illegible] ".
  - Este [illegible] nfigurações e avalia [illegible]




```
import ma
from skle

training_
test_accu

# tentand
h = range
for i in
    # Con
    class


    # Tre
    class
    # res
    train                                                    lasse_treinamento))
    # res
    test_accuracy.append(classificador.score(previsores_teste, classe_teste))

plt.plot(h, training_accuracy, label="training accuracy")
plt.plot(h, test_accuracy, label="test accuracy")
plt.ylabel("Accuracy")
plt.xlabel("K")
plt.legend()
```

- **Construindo o script**
  - Para descobrir a melhor configuração do algoritmo, crie um novo arqui [illegible] ".
  - Este [illegible] nfigurações e avalia [illegible]




```
import ma
from skle

training_
test_accu

# tentand
h = range
for i in
    # Cor
    class

    # Tre
    class
    # res
    train                                                    lasse_treinamento))
    # res
    test_accuracy.append(classificador.score(previsores_teste, classe_teste))

plt.plot(h, training_accuracy
plt.plot(h, test_accuracy, la
plt.ylabel("Accuracy")
plt.xlabel("K")
plt.legend()
```

Neste caso, um K=16 parece ser a melhor decisão.

- **Construindo o script**

  – Voltando no script da árvore de decisão, podemos colocar o valor de K e coletar os resultados.

```
############## Classificação com árvores de decisão ##########

import pandas as pd
from sklearn.tree import DecisionTreeClassifier

# gereção da árvore
classificador = DecisionTreeClassifier(criterion='entropy',
                                       max_depth=16,
                                       random_state=0)

#  Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

# teste
previsoes = classificador.predict(previsores_teste)

#análise de resultados
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste,previsoes)
matriz_teste = confusion_matrix(classe_teste,previsoes)



#  Exportando a arvore
from sklearn.tree import export_graphviz
export_graphviz(classificador,
                out_file="tree.dot",
                class_names=["bad", "good"],
                feature_names=cols_previsores,
                impurity=False,
                filled=True)


#  Visualizando a arvore
import graphviz
with open("tree.dot") as f:
    dot_graph = f.read()
display(graphviz.Source(dot_graph))
```

- **Construindo o script**

  – Voltando no script da árvore de decisão, podemos colocar o valor de K e coletar os resultados.

```python
############## Classificação com árvores de decisão ##########

import pandas as pd
from sklearn.tree import DecisionTreeClassifier

# gereção da árvore
classificador = DecisionTreeClassifier(criterion='entropy',
                                       max_depth=16,
                                       random_state=0)
```

acuracia_teste	float	1	0.6333333333333333
matriz_teste	Array of int64	(2, 2)	[[44 26] [29 51]]

```python
#análise de resultados
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste,previsoes)
matriz_teste = confusion_matrix(classe_teste,previsoes)



#  Exportando a arvore
from sklearn.tree import export_graphviz
export_graphviz(classificador,
                out_file="tree.dot",
                class_names=["bad", "good"],
                feature_names=cols_previsores,
                impurity=False,
                filled=True)


#  Visualizando a arvore
import graphviz
with open("tree.dot") as f:
    dot_graph = f.read()
display(graphviz.Source(dot_graph))
```

Precisamos registrar este resultado para futuras comparações!

- **Registrando os resultados**

  – O arquivo atualizado com os resultados para esta base fica assim:

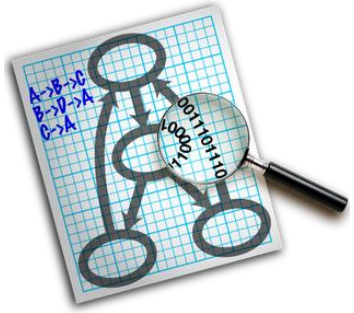
Dados de Crédito	
Descrição	Nesse conjunto de dados, cada entrada representa uma pessoa que recebe um crédito por um banco. Cada pessoa é classificada como risco de crédito bom ou ruim, de acordo com o conjunto de atributos.
Total de registros:	1000
Total de características:	9
Total de colunas de classes:	1

Resultados

Método	Configuração	Tratamento de dados	Acurácia %	Matriz de Confusão	
Classe Majoritária	-	-	0.4667	70 80	0 0
KNN	K=5 Distancia Euclidiana	Criação de variáveis dummy	0.5533	41 38	29 42
KNN	K=29 Distancia Euclidiana	Criação de variáveis dummy	0.5867	43 35	27 45
KNN	K=30 Distancia Euclidiana	Criação de variáveis dummy Padronização	0.6333	49 34	21 46
Naive Bayes	-	Apenas LabelEncoder	0.56	38 34	32 46
Naive Bayes	-	Apenas LabelEncoder Padronização	0.5533	38 35	32 45
Naive Bayes	-	Apenas LabelEncoder Sem coluna [‘Duration’]	0.52	39 41	31 39
Árvores de Decisão	criterion='entropy', max_depth=16, random_state=0	Apenas LabelEncoder Sem coluna [‘Duration’]	0.6333	44 29	26 51

Árvores de Decisão

- **Visualização da árvore**
  - Como foi dito, uma das vantagens das Árvores de Decisão é que elas permitem uma **análise interna** de como a classificação está sendo feita.
  - Em outras palavras, queremos saber quais são as **características mais importantes** da base e como elas influenciam no resultado final.
  - Para isso, precisamos de uma ferramenta de **visualização de grafos**:



Graphviz

- **Instalação do Graphviz**

  – Passo-a-passo:

1. Vá para o site Graphviz, faça o download e instale no seu computador.
2. Adicione o Graphviz à variável de ambiente "Path":
   - Vá para Computador→Propriedades→Configurações avançadas do sistema→Variáveis de ambiente e encontre “Path" na caixa de variáveis do sistema. Clique no caminho e clique em editar:
   - Anexe: “C:\Arquivos de Programas (x86)\Graphviz2.38\bin” ao final dos muitos caminhos que já estão presentes no Path.
3. Para verificar a instalação, vá para o prompt de comando e digite: “dot –V”. isso deve mostrar a versão do Graphviz instalada.
4. Vá para o prompt de comando do Anaconda e digite: “pip install graphviz”
5. Reinicie o Spyder ou inicie-o se ainda não estiver aberto.

- **Construindo o script**

  – Voltando ao script da árvore de decisão, agora você pode executar as últimas linhas para salvar e exibir a árvore.

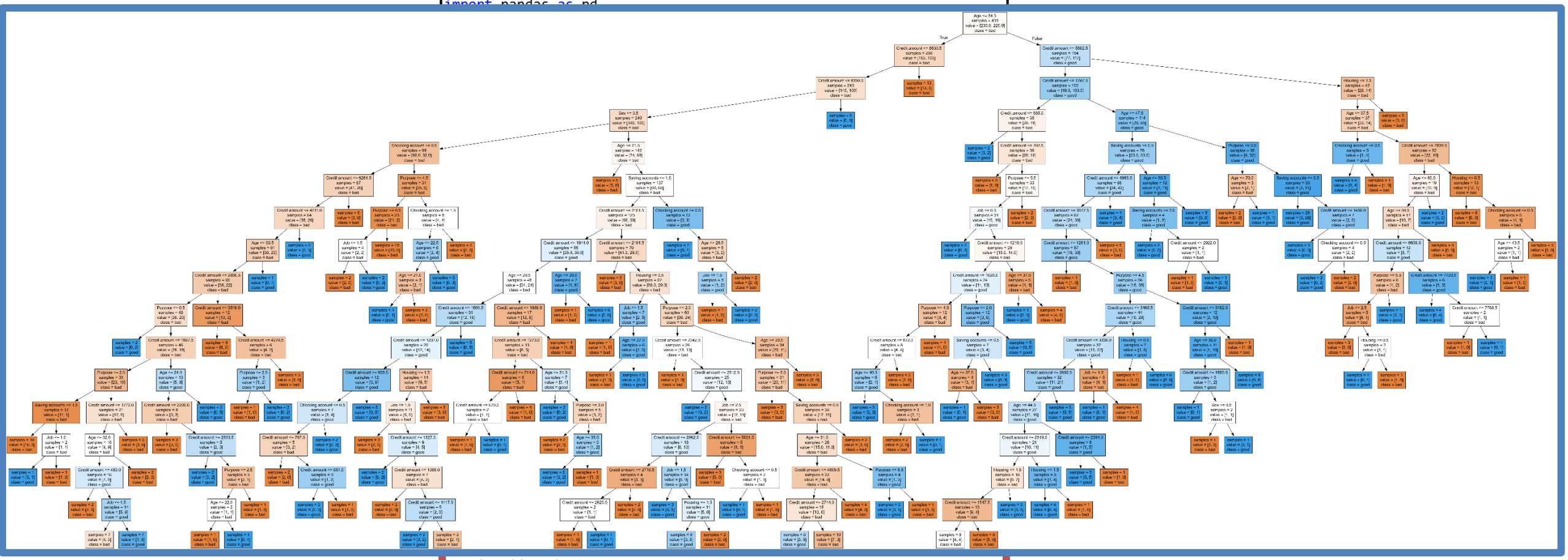
```python
############## Classificação com árvores de decisão ##########

import pandas as pd
from sklearn.tree import DecisionTreeClassifier

# gereção da árvore
classificador = DecisionTreeClassifier(criterion='entropy',
                                       max_depth=16,
                                       random_state=0)

#  Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

# teste
previsoes = classificador.predict(previsores_teste)

#análise de resultados
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste,previsoes)
matriz_teste = confusion_matrix(classe_teste,previsoes)


#  Exportando a arvore
from sklearn.tree import export_graphviz
export_graphviz(classificador,
                out_file="tree.dot",
                class_names=["bad", "good"],
                feature_names=cols_previsores,
                impurity=False,
                filled=True)


#  Visualizando a arvore
import graphviz
with open("tree.dot") as f:
    dot_graph = f.read()
display(graphviz.Source(dot_graph))
```

A árvore está salva no arquivo “tree.dot”.
Para convertê-la de “.dot” para “.png”, execute o script “converter_dot_png.bat”

- **Construindo o script**

  – Voltando ao script da árvore de decisão, agora você pode executar as últimas linhas para salvar e exibir a árvore.

```
############## Classificação com árvores de decisão ##########

import pandas as pd
```



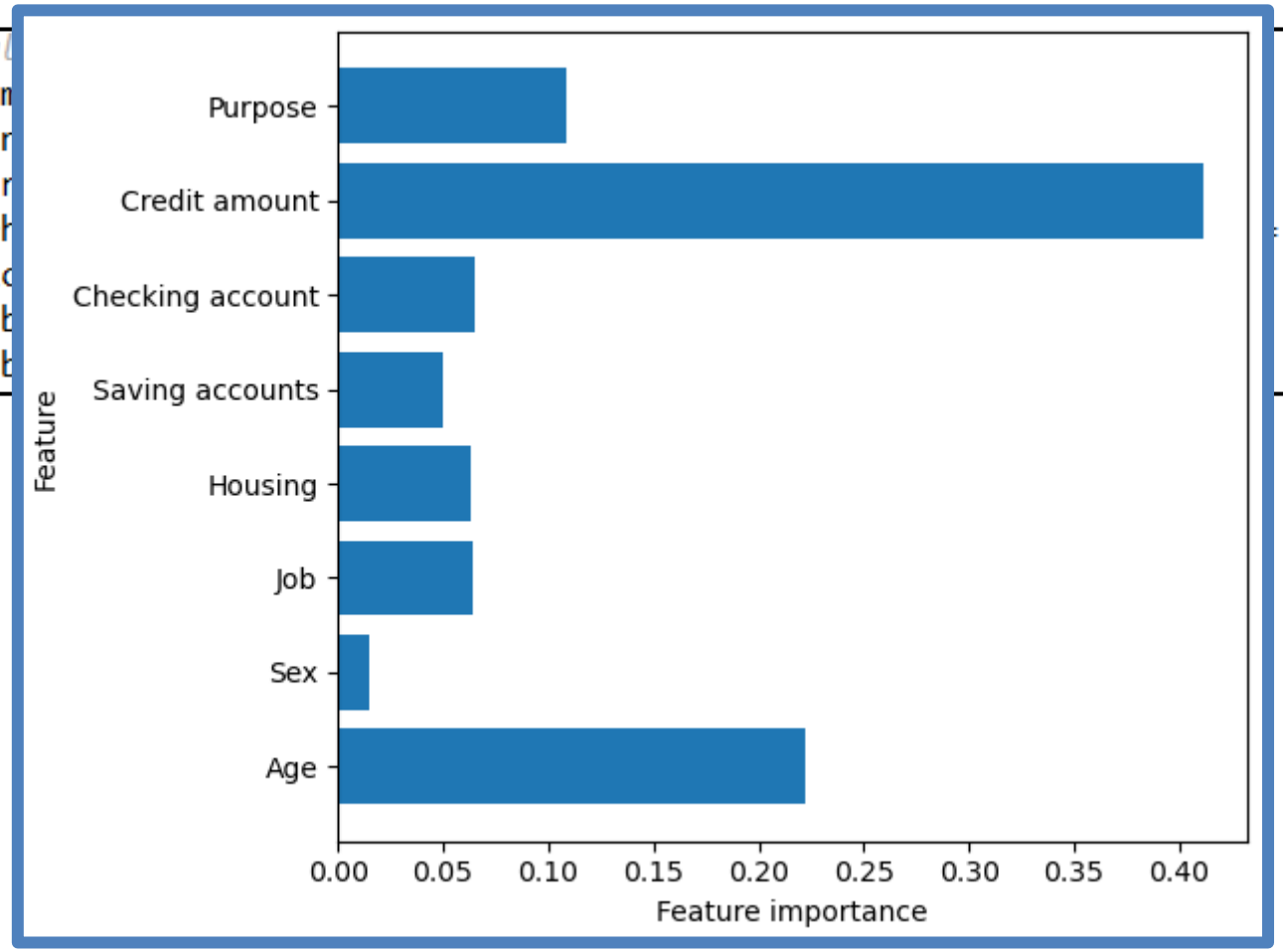
```
#  Visualizando a arvore
import graphviz
with open("tree.dot") as f:
    dot_graph = f.read()
display(graphviz.Source(dot_graph))
```

- **Construindo o script**
  - Além disso, é interessante gerar um gráfico com a **importância** que cada característica da base teve no resultado final.
  - Para isso, acrescente as seguintes linhas ao final do script:

```
# Visualizando a importância das características
import matplotlib.pyplot as plt
import numpy as np
n_features = previsores.columns.size
plt.barh(range(n_features), classificador.feature_importances_, align='center')
plt.yticks(np.arange(n_features), previsores.columns)
plt.xlabel("Feature importance")
plt.ylabel("Feature")
```

- **Construindo o script**
  - Além disso, é interessante gerar um gráfico com a **importância** que cada característica da base teve no resultado final.
  - Para isso, acrescente as seguintes linhas ao final do script:

```
# Visual
import m
import r
n_featur
plt.barh
plt.ytic
plt.xlab
plt.ylab
```

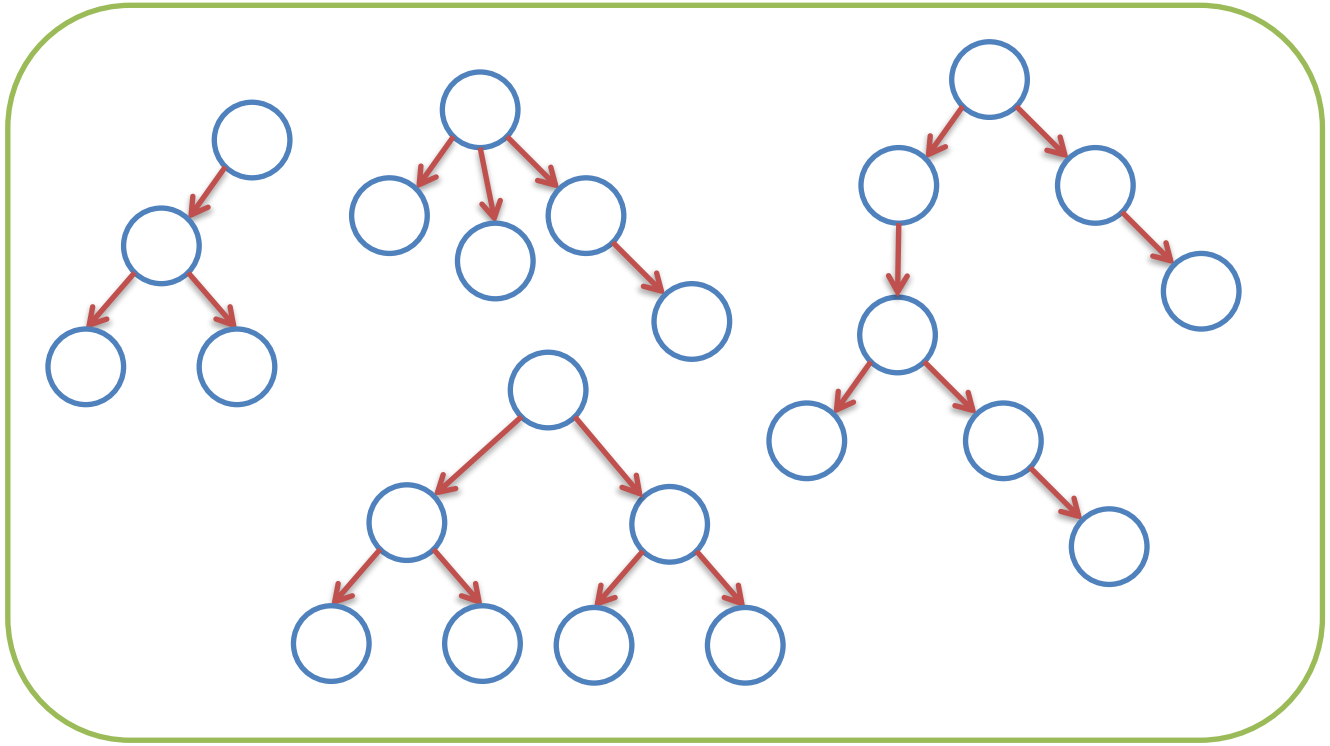



```
'center')
```

Random Forest

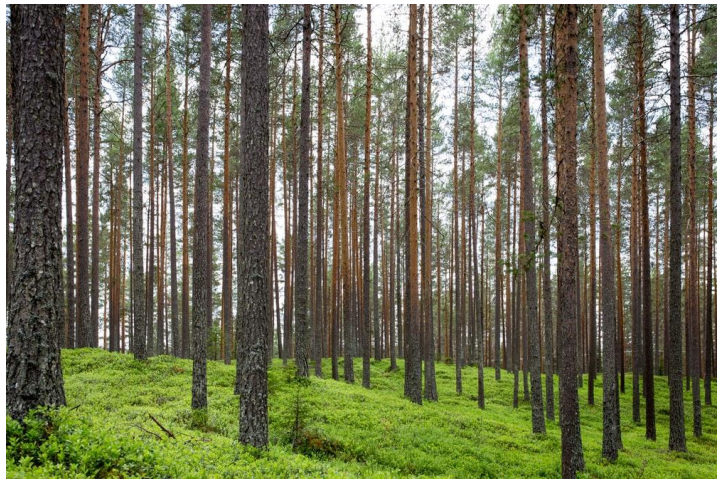
Random Forest

- **Introdução**
  - Floresta é um conjunto de árvores.



Random Forest

- **Introdução**
  - Uma estratégia muito usada para **melhorar os resultados** das árvores de decisão é utilizar **mais de uma** árvore de decisão.
  - Esta estratégia tem se mostrado **eficaz** em muitas aplicações.
  - Uma **floresta randômica** é basicamente um conjunto de árvores de decisão levemente diferentes umas das outras.



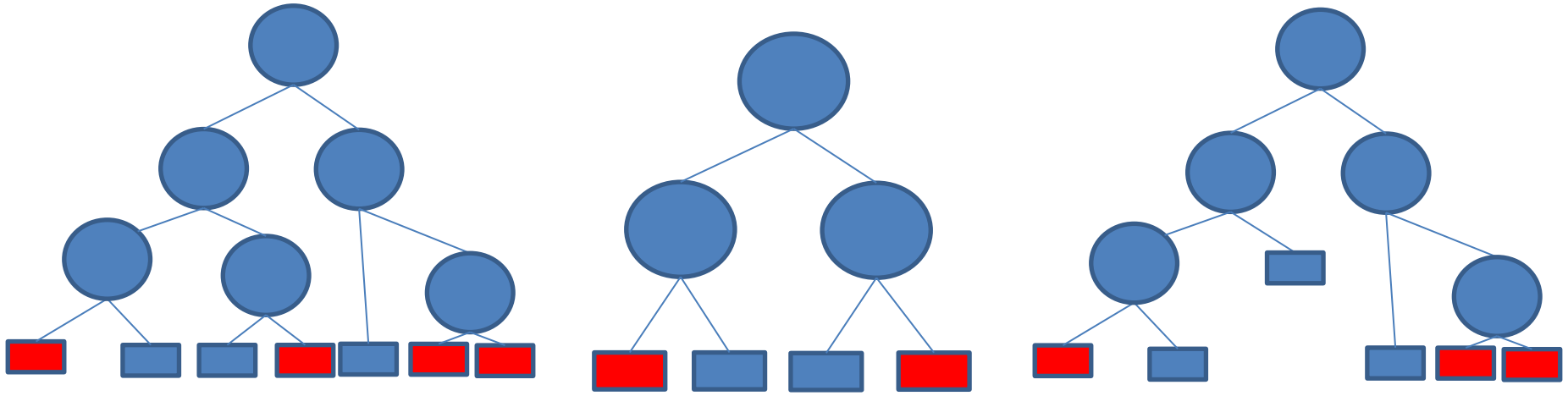
Random Forest

- **Descrição da técnica**
  - As árvores de uma floresta randômica são geradas adicionando-se um pouco de **aleatoriedade** em sua construção.
  - Normalmente, existem **duas maneiras** de inserir a aleatoriedade:
    - Na seleção aleatória de **partições dos registros** para a construção da árvore.
    - Na seleção aleatória de **características usadas** para cada split.
  - O **número de árvores** na floresta é um dos **parâmetros** do algoritmo.

Random Forest

História do crédito	Dívida	Garantias	Renda anual	Risco
Ruim	Alta	Nenhuma	< 15.000	Alto
Desconhecida	Alta	Nenhuma	>= 15.000 a <= 35.000	Alto
Desconhecida	Baixa	Nenhuma	>= 15.000 a <= 35.000	Moderado
Desconhecida	Baixa	Nenhuma	> 35.000	Alto
Desconhecida	Baixa	Nenhuma	> 35.000	Baixo
Desconhecida	Baixa	Adequada	> 35.000	Baixo
Ruim	Baixa	Nenhuma	< 15.000	Alto
Ruim	Baixa	Adequada	> 35.000	Moderado
Boa	Baixa	Nenhuma	> 35.000	Baixo
Boa	Alta	Adequada	> 35.000	Baixo
Boa	Alta	Nenhuma	< 15.000	Alto
Boa	Alta	Nenhuma	>= 15.000 a <= 35.000	Moderado
Boa	Alta	Nenhuma	> 35.0000	Baixo
Ruim	Alta	Nenhuma	>= 15.000 a <= 35.000	Alto

- **Descrição da técnica**
  - A ideia seria consultar **diversos especialistas** antes antes de tomar uma decisão.
  - No final, a classe seria a **mais votada** pelas árvores da floresta.



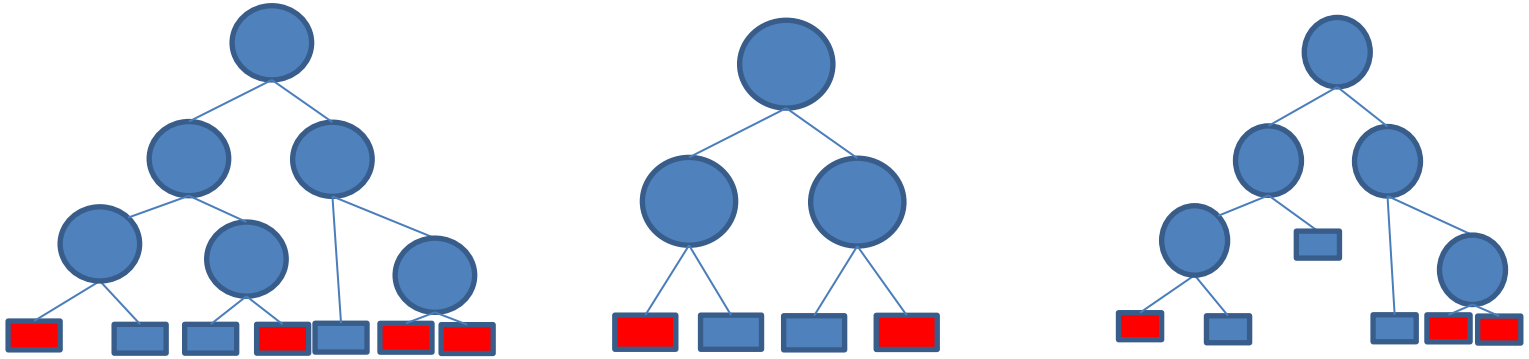
Risco = Alto

Risco = Baixo

Risco = Baixo

Random Forest

- **Descrição da técnica - seleção aleatória de características**
  - A técnica irá escolher de forma aleatória **K atributos** para comparação da métrica de pureza (entropia/gini).
  - Com este número de características serão geradas **N árvores**.
  - K e N são **parâmetros de configuração** da técnica.



História de crédito
Dívida
Garantias

Renda
Dívida
Garantias

Renda
História de crédito
Dívida

- **Construindo o script**

  – Primeiramente, o script de pré-processamento deve ser executado, selecionando quais as transformações precisam ser feitas na base.

Spyder (Python 3.12)

Arquivo Editar Pesquisar Código Executar Depurar Consoles Projetos Ferramentas Ver Ajuda

E:\PythonWorkspace\2025.1\aula4

...tual\2025-1_Técnicas de Aprendizado de Máquina (reformulação)\aula 3\Programas\pre-processamento.py

pre-processamento.py × credit_data.csv ×

```python
# -*- coding: utf-8 -*-
"""
Created on Mon Mar 31 18:32:59 2025

@author: marco
"""

import pandas as pd
base = pd.read_csv('credit_data.csv')
resumo = base.describe()


# ==========================================================================
#                        Tratando valores inválidos
# ==========================================================================

# Verifica idades inválidas
base.loc[base['Age'] < 0]

# Apagar a coluna
# base.drop('Age', axis=1, inplace=True)

# Apagar apenas os registros com problema
# base.drop(base[base['Age'] < 0].index, inplace=True)

# Preencher os valores com a média
idade_media = base['Age'][base['Age'] > 0].mean().round()
base.loc[base['Age'] < 0, 'Age'] = idade_media

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# Preencher os valores com o mais frequente
saving_maioria = base['Saving accounts'][pd.notnull(base['Saving accounts'])]
```

Nome	Tipo	Tamanho	Valor
base	DataFrame	(1000, 11)	Column names: Id, Age, …
checking_maioria	str	6	little
classe	DataFrame	(600, 1)	Column names: Risk
classe_teste	DataFrame	(150, 1)	Column names: Risk
classe_treinamento	DataFrame	(450, 1)	Column names: Risk
cols_classe	list	1	['Risk']
cols_previsores	Index	(18,)	Index: 18 entries, Age …

Ajuda Explorador de variáveis Figuras Arquivos

Console 1/A ×

```
Python 3.12.4 | packaged by Anaconda, Inc. | (main, Jun 18 2024,
15:03:56) [MSC v.1929 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 8.25.0 -- An enhanced Interactive Python.

In [1]:
   ...: """
   ...: Created on Mon Mar 31 18:32:59 2025
   ...:
   ...: @author: marco
   ...: """
   ...:
   ...: import pandas as pd
   ...: base = pd.read_csv('credit_data.csv')
```

Console IPython Histórico

conda: base (Python 3.12.4) Completions: conda(base) LSP: Python Line 3, Col 36 UTF-8 CRLF RW Mem 75%

- **Construindo o script**
  - Para construir as Random Forests, crie um novo arquivo chamado “random_forest.py”.
  - Nele, insira o seguinte código:

```
############## Classificação com random forest ##########

import pandas as pd
from sklearn.ensemble import RandomForestClassifier

# gereção da árvore
classificador = RandomForestClassifier(criterion='gini',
                                       max_depth=15,
                                       n_estimators=9,
                                       max_features=7,
                                       random_state=0)

#  Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

# teste
previsoes = classificador.predict(previsores_teste)

#análise de resultados
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste,previsoes)
matriz_teste = confusion_matrix(classe_teste,previsoes)
```

- **Construindo o script**
  - Para construir as Random Forests, crie um novo arquivo chamado "random_forest.py".
  - Nele, insira o seguinte código:

```
############## Classificação com random forest ##########

import pandas as pd
from sklearn.ensemble import RandomForestClassifier

# gereção da árvore
classificador = RandomForestClassifier(criterion='gini',
                                       max_depth=15,
                                       n_estimators=9,
                                       max_features=7,
                                       random_state=0)

#  Treinamento
classificador.fit(previsores_treinamento, classe_treinamento)

# teste
previsoes = classificador.predict(previsores_teste)

#análise de resultados
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste,previsoes)
matriz_teste = confusion_matrix(classe_teste,previsoes)
```

Muitos parâmetros precisam ser configurados e testados.

- **Construindo o script**

  – Voltando no script da árvore de decisão, podemos colocar o valor de K e coletar os resultados.

```
############## Classificação com random forest ##########

import pandas as pd
from sklearn.ensemble import RandomForestClassifier
```

acuracia_teste	float	1	0.6533333333333333
matriz_teste	Array of int64	(2, 2)	[[40 30] [22 58]]

```
                                    n_estimators=9,
                                    max_features=7,
                                    random_state=0)

#  Treinamento
classificador.fit(previsores_treinamento, cl

# teste
previsoes = classificador.predict(previsores

#análise de resultados
from sklearn.metrics import confusion_matrix, accuracy_score
acuracia_teste = accuracy_score(classe_teste,previsoes)
matriz_teste = confusion_matrix(classe_teste,previsoes)
```

Precisamos registrar este resultado para futuras comparações!

- **Registrando os resultados**

  – O arquivo atualizado com os resultados para esta base fica assim:

Dados de Crédito	
Descrição	Nesse conjunto de dados, cada entrada representa uma pessoa que recebe um crédito por um banco. Cada pessoa é classificada como risco de crédito bom ou ruim, de acordo com o conjunto de atributos.
Total de registros:	1000
Total de características:	9
Total de colunas de classes:	1

Resultados

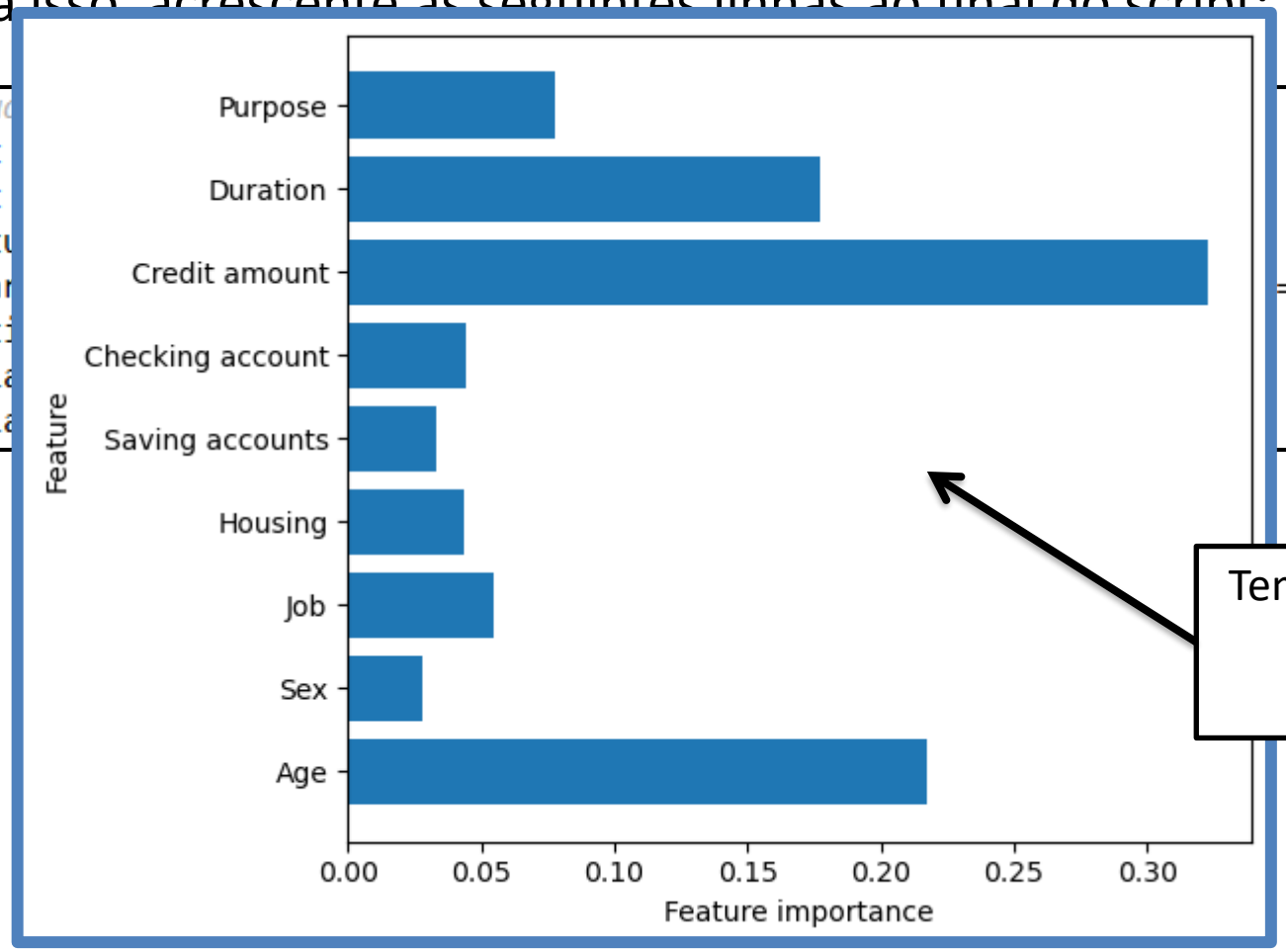
Método	Configuração	Tratamento de dados	Acurácia %	Matriz de Confusão
Classe Majoritária	-	-	0.4667	70 0 80 0
KNN	K=5 Distancia Euclidiana	Criação de variáveis dummy	0.5533	41 29 38 42
KNN	K=29 Distancia Euclidiana	Criação de variáveis dummy	0.5867	43 27 35 45
KNN	K=30 Distancia Euclidiana	Criação de variáveis dummy Padronização	0.6333	49 21 34 46
Naive Bayes	-	Apenas LabelEncoder	0.56	38 32 34 46
Naive Bayes	-	Apenas LabelEncoder Padronização	0.5533	38 32 35 45
Naive Bayes	-	Apenas LabelEncoder Sem coluna [‘Duration’]	0.52	39 31 41 39
Árvores de Decisão	criterion='entropy', max_depth=16, random_state=0	Apenas LabelEncoder Sem coluna [‘Duration’]	0.6333	44 26 29 51
Random Forest	criterion='gini', max_depth=15, n_estimators=9, max_features=7, random_state=0	Apenas LabelEncoder	0.6533	40 30 22 58

- **Construindo o script**
  - Apesar de não ser possível **visualizar** as **árvores**, aqui também podemos gerar um **gráfico** com a importância que cada característica da base teve no resultado final.
  - Para isso, acrescente as seguintes linhas ao final do script:

```
# Visualizando a importância das características
import matplotlib.pyplot as plt
import numpy as np
n_features = previsores.columns.size
plt.barh(range(n_features), classificador.feature_importances_, align='center')
plt.yticks(np.arange(n_features), previsores.columns)
plt.xlabel("Feature importance")
plt.ylabel("Feature")
```

- **Construindo o script**
  - Apesar de não ser possível **visualizar** as **árvores**, aqui também podemos gerar um **gráfico** com a importância que cada característica da base teve no resultado final.
  - Para isso, acrescente as seguintes linhas ao final do script:

```
# Visu
import
import
n_featu
plt.bar
plt.yti
plt.xla
plt.yla
```




```
='center')
```

Temos muito mais informação sobre a relevância das características!

FIM